1.1 Introduction to Project Theme:

The project **Web Application for Dental Clinic Appointment System** is a web-based application created to make the daily work of a dental clinic easier and more organized. The main purpose of this system is to help patients book appointments online and to help the clinic manage appointments and patient information digitally.

In many clinics, appointments are still handled using phone calls or notebooks. This method is time-consuming and can cause problems like lost records, missed appointments, and no proper reminders for patients. This project solves these problems by providing an online system where patients can book appointments easily and receive automatic email reminders.

The system is designed to:

- Save time by reducing manual work.
- Improve patient experience with online appointment booking.
- Help the clinic manage data in a simple and organized way.

This project uses modern web technologies to build a simple and useful system that improves communication between patients and the clinic while making clinic operations more efficient.

1.2 Working of System:

The Dental Clinic Appointment System is designed to simplify the process of managing appointments, patient records, and automated reminders. The system works through different modules that interact with each other to ensure smooth clinic operations.

1.2.1 Patient Side Working

1. The patient visits the website.
2. The patient can:
   - ✓ View available dental services.
   - ✓ Book an appointment by filling in personal details, preferred date, and time.
   - ✓ Submit contact or inquiry forms.
3. After submitting the appointment form:
   - ✓ The system stores the details in the database.
   - ✓ A confirmation message is shown to the patient.
4. Before the appointment:
   - ✓ The system automatically sends reminder emails (24 hours and 4 hours before the appointment).

1.2.2 Admin Side Working

1. The admin logs into the admin panel using secure credentials.
2. After login, the admin can:
   - ✓ View all appointments.
   - ✓ Filter appointments by date or status.
   - ✓ Update appointment status (Pending, Confirmed, Completed, Cancelled).
   - ✓ Manage services offered by the clinic.
   - ✓ View contact messages and patient inquiries.
3. All updates made by the admin are stored in the database instantly.

1.3 Need & Scope of Proposed System:

Need of the Proposed System –

The need for the proposed system arises from the increasing demand for digital solutions in healthcare services. Managing a dental clinic using manual methods is time-consuming, inefficient, and prone to errors. Therefore, there is a strong need for a system that can automate appointment booking, improve patient communication, and manage clinic data in a structured and reliable manner.

The proposed system provides an online platform that allows patients to book appointments easily without visiting the clinic or making phone calls. This saves time for both patients and clinic staff. The system also generates a unique appointment ID for every booking, which helps patients track their appointment status and access reports.

One of the major needs of this system is to introduce **automation** in daily clinic operations. By using automated email reminders, the system ensures that patients are informed about their appointments in advance. This reduces missed appointments and improves overall clinic efficiency.

The proposed system is designed to fulfil the following needs:

- Reduce manual work for clinic staff
- Provide easy online appointment booking
- Improve communication with patients
- Maintain digital and secure patient records
- Send automatic appointment reminders
- Improve time management and service quality

Scope of the Proposed System -

The scope of the proposed system defines what the system can do and what features are included in the project. This system focuses on the core functionalities required for a dental clinic.

The scope includes:

- Online appointment booking system
- Service selection by patients
- Unique appointment ID generation
- Admin panel for clinic staff
- Email-based reminder system
- Contact and feedback management
- Uploading and viewing patient reports
- Basic clinic settings management

The proposed system is mainly designed for small and medium dental clinics. It is scalable and can be extended in the future by adding features such as SMS reminders, online payments, and patient login system.

1.4 Organization Profile:

The **Aabha Dental Clinic** is a healthcare organization that provides professional dental care services to patients. The clinic is committed to delivering high-quality treatment, maintaining hygiene standards, and ensuring patient comfort and satisfaction.

The clinic offers various dental services such as general check-ups, teeth cleaning, cavity treatment, root canal treatment, tooth extraction, cosmetic dentistry, and other specialized dental procedures. The clinic is managed by qualified dental professionals and supported by trained staff to ensure smooth operations.

The main objective of the clinic is to provide safe, affordable, and reliable dental care while maintaining proper patient records and appointment scheduling. The clinic focuses on improving patient experience through timely service and clear communication.

With the increasing number of patients and appointments, the clinic requires a digital system to manage appointments, patient information, and reminders efficiently. The Dental Clinic Appointment System helps the organization reduce manual work, improve accuracy, and enhance overall productivity.

The organization believes in adopting modern technology to streamline operations and provide better healthcare services to patients.

2.1 Objectives of the System

The main objectives of the Dental Clinic Appointment System are:

1. To automate appointment booking and reduce manual paperwork.
2. To manage patient records efficiently in a secure digital database.
3. To provide an easy interface for admin to view, update, and manage appointments.
4. To send automatic reminders (24 hours and 4 hours before appointment) to reduce missed appointments.
5. To improve communication between patients and the clinic through email notifications.
6. To allow filtering and reporting of appointments based on date and status.
7. To ensure data accuracy and security using proper authentication and database management.
8. To reduce time consumption and human errors in scheduling and record maintenance.
9. To enhance overall clinic efficiency and patient satisfaction.

2.2 Software Requirement Specification:

1. Functional Requirements

1.1 Patient Module

- The system shall allow patients to view available dental services.
- The system shall allow patients to book an appointment.
- The system shall store patient details such as name, contact number, email, date, and time.
- The system shall display confirmation after booking.

1.2 Admin Module

- The system shall allow admin login using secure authentication.
- The system shall allow admin to view all appointments.
- The system shall allow filtering appointments by date and status.
- The system shall allow admin to update appointment status (Pending, Confirmed, Completed, Cancelled).
- The system shall allow admin to manage services (Add, Edit, Delete).

1.3 Reminder System

- The system shall automatically send reminder emails 24 hours before the appointment.
- The system shall automatically send reminder emails 4 hours before the appointment.
- The system shall prevent duplicate reminders.

1.4 Contact Module

- The system shall allow patients to send messages or inquiries.
- The system shall store contact messages in the database.
- The admin shall be able to view submitted messages.

2. Non-Functional Requirements

These describe how the system should perform.

2.1 Performance Requirements

- The system should respond quickly to user requests.
- The system should handle multiple appointment bookings efficiently.

2.2 Security Requirements

- The system should use authentication for admin login.
- The system should protect patient data in the database.
- The system should prevent unauthorized access.

2.3 Reliability Requirements

- The system should store data accurately without loss.
- The reminder system should run automatically at scheduled intervals.

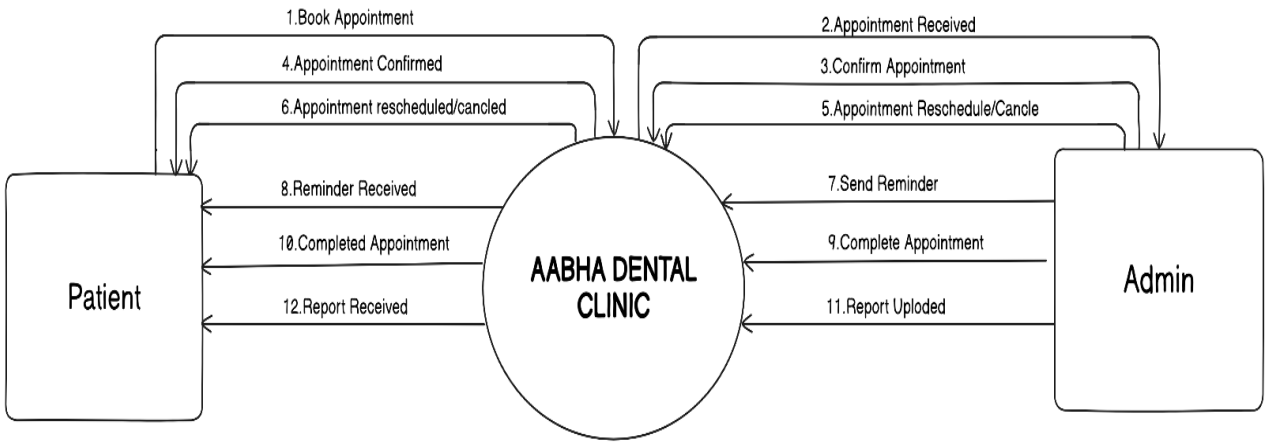
2.4 Usability Requirements

- The system should have a simple and user-friendly interface.
- The system should be easy to navigate for both patients and admin.

2.5 Compatibility Requirements

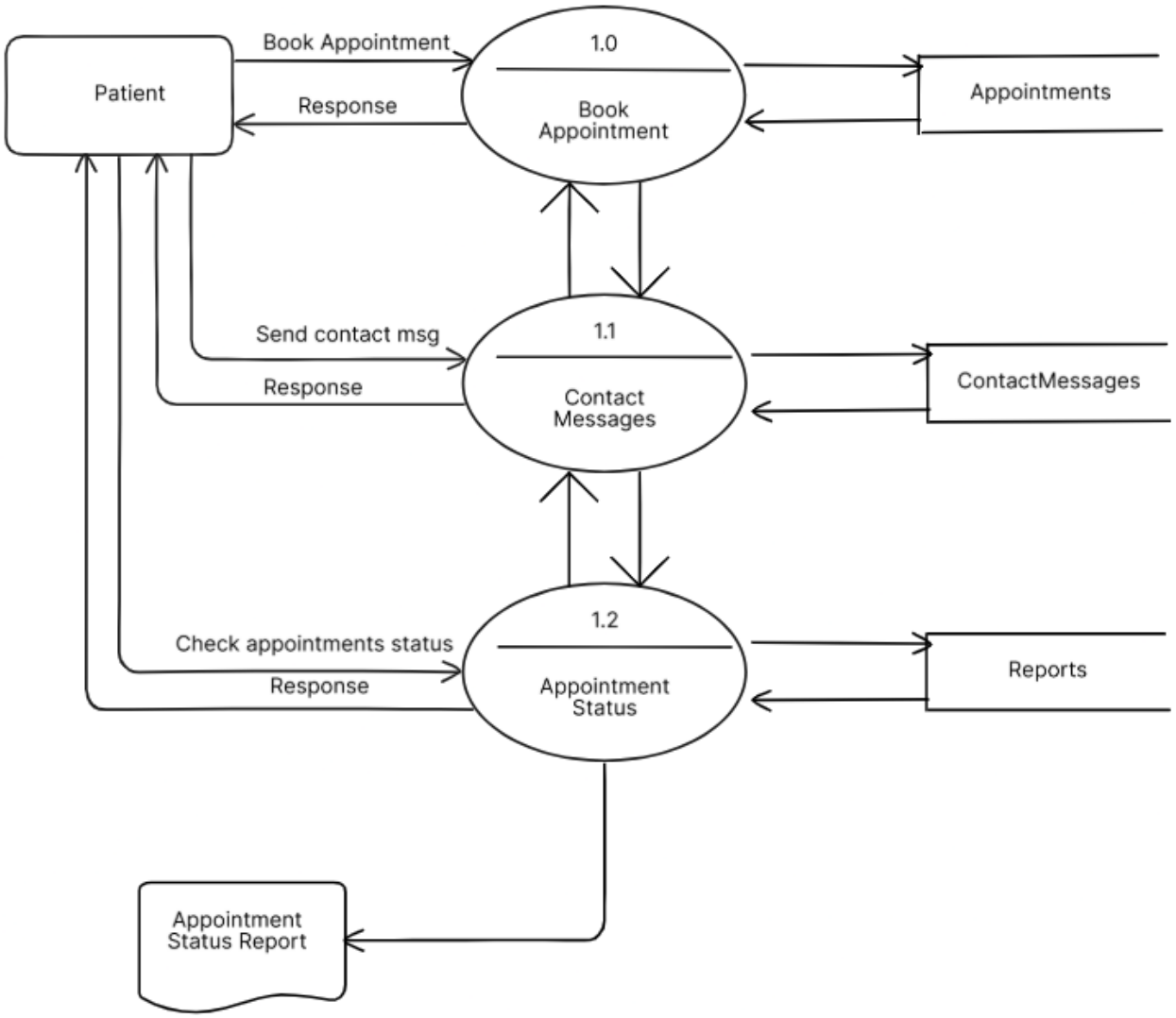
- The system should work on modern web browsers.
- The system should be responsive for desktop and mobile devices.

3.1 Context Level diagram:

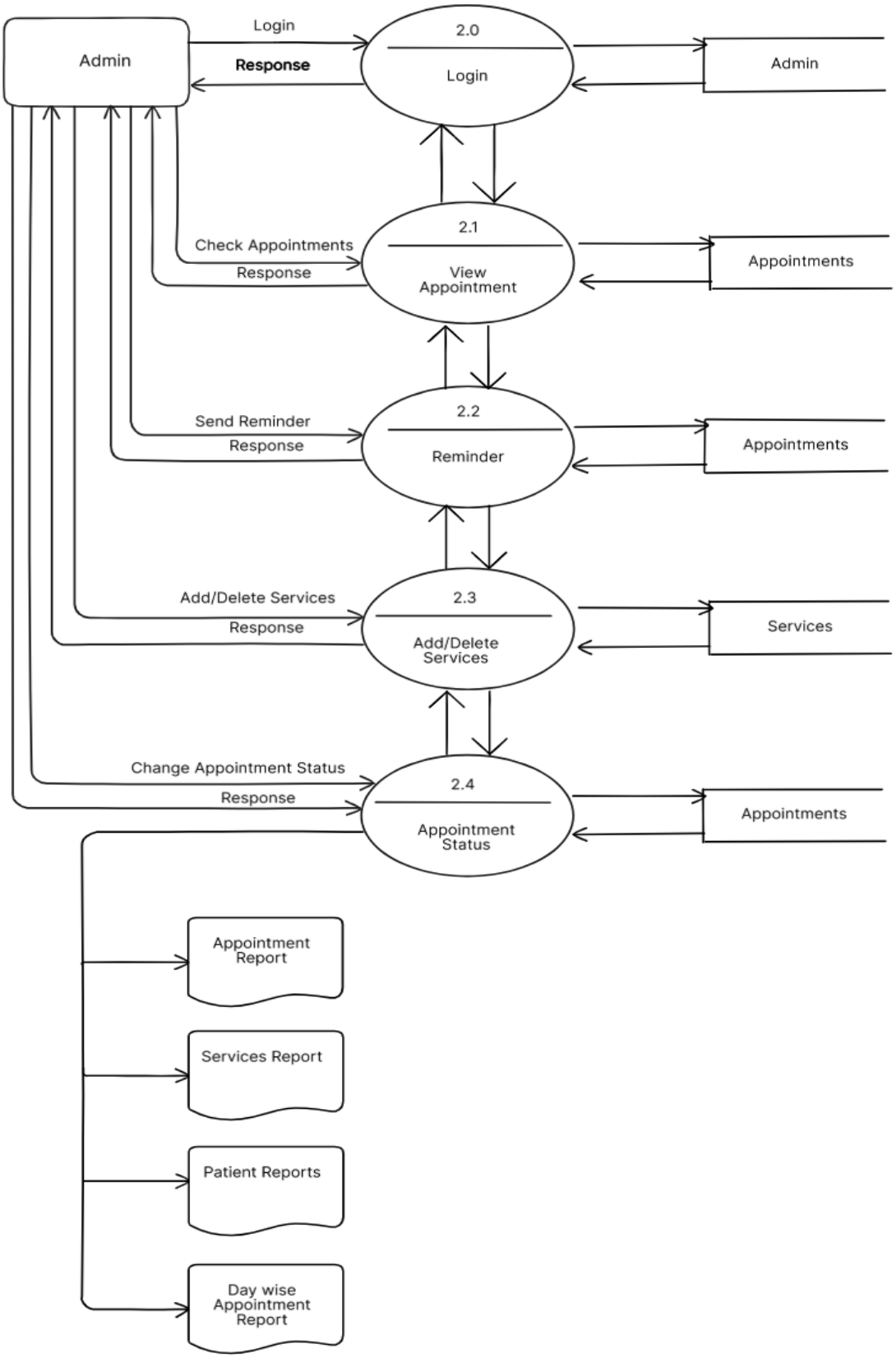


3.2 DFD

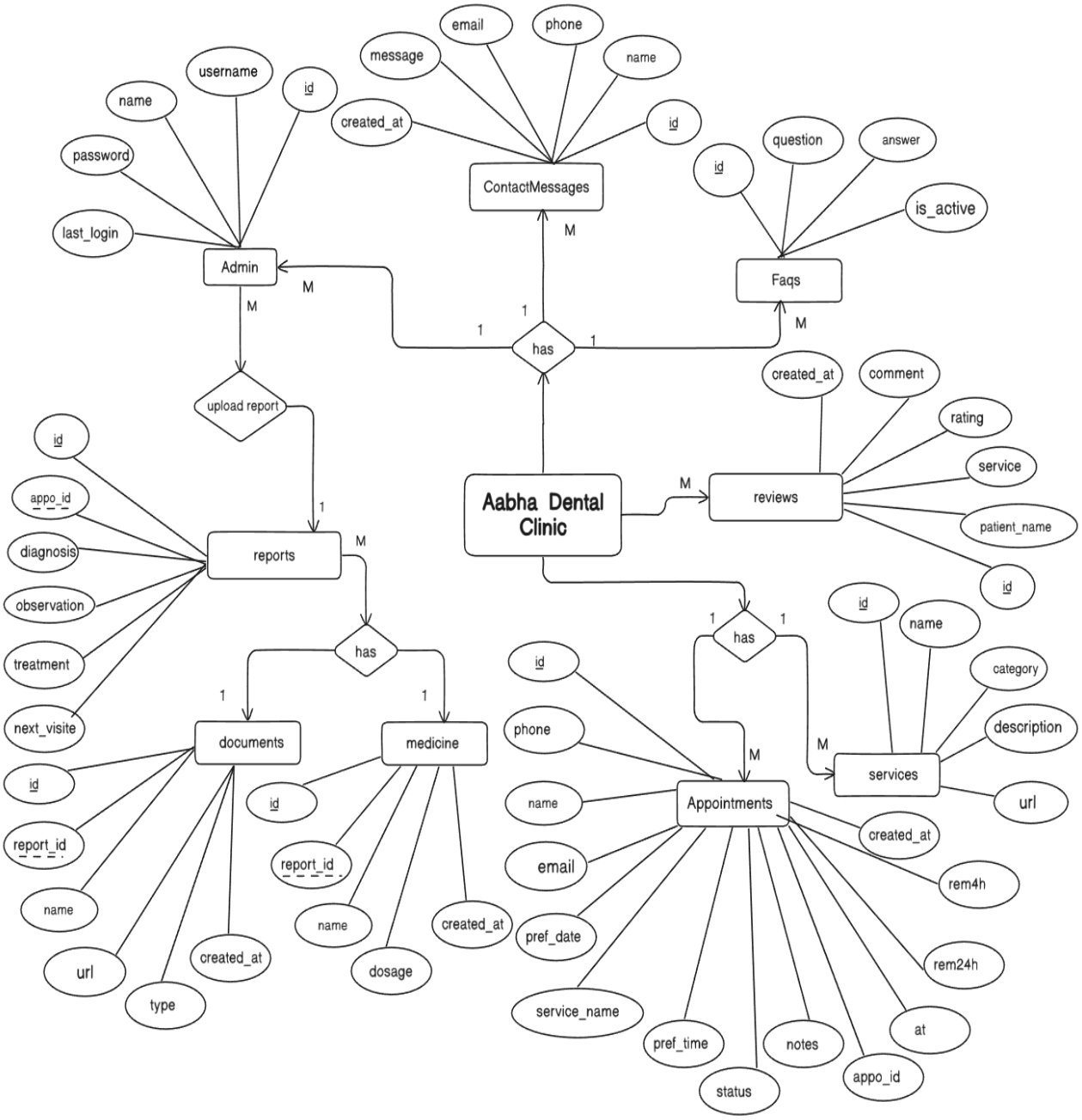
Patient:



Admin:


Admin
Login
Response
2.0
Login
Admin
Check Appointments
Response
2.1
View Appointment
Appointments
Send Reminder
Response
2.2
Reminder
Appointments
Add/Delete Services
Response
2.3
Add/Delete Services
Services
Change Appointment Status
Response
2.4
Appointment Status
Appointments
Appointment Report
Services Report
Patient Reports
Day wise Appointment Report

3.3 ERD:


email
phone
message
name
username
id
name
created_at
id
password
ContactMessages
question
answer
id
last_login
is_active
Admin
M
Faqs
M
M
1
1
has
1
M
created_at
comment
upload report
rating
id
service
appo_id
Aabha Dental Clinic
M
reviews
1
patient_name
diagnosis
reports
M
id
observation
id
name
1
has
1
treatment
has
id
category
1
1
next_visite
documents
medicine
phone
description
M
M
services
id
name
Appointments
url
id
created_at
report_id
report_id
email
rem4h
name
name
created_at
pref_date
rem24h
dosage
url
created_at
type
service_name
at
pref_time
notes
appo_id
status

3.4 System Requirements:

3.4.1 Hardware Requirement:

Component	Minimum Requirement
Processor	Intel Core i3 or above
RAM	4 GB or more
Hard Disk	128 GB free space
Monitor	1024x768 resolution minimum
Network	Internet connection for online access(must)

3.4.2 Software Requirements:

Component	Requirement
Operating System	Windows/Linux/MacOS
Front-End Technologies	React.js, Next.js
Back-End Language	Node.js (Next.js API Routes)
Web Server	Vercel (Cloud hosting) / Localhost
Database	PostgreSQL
Browser	Chrome, Firefox
Email Service	Nodemailer (Gmail SMTP)
Development Tools	VS Code, Git, GitHub

4.1Database Design (Data Dictionary):

1. Table Name: Admin

Sr_No	Field Name	Data Type	Description
1	Id(PK)	serial	Store admin id
2	Username	varchar(25)	Store username
3	Password_hash	varchar(255)	Store password
4	Last_login	timestamp	Store last login time

2. Table Name: Appointments

Sr_No	Field Name	Data Type	Description
1	id(PK)	serial	Store record id
2	name	varchar(255)	Store patient name
3	phone	varchar(11)	Store patient phone no
4	email	varchar(255)	Store patient email
5	service_name	varchar(255)	Store service name
6	preferred_date	varchar(255)	Store Appointment date
7	preferred_time	varchar(255)	Store Appointment time
8	status	varchar(50)	Store appointment status
9	notes	varchar(255)	Store extra notes
10	created_at	timestamp	Store creation date
11	appointment_id	varchar(255)	Store appointment id
12	at	varchar(255)	Store clinic location
13	reminder24h	Boolean	Store reminder sent or not
14	reminder4h	Boolean	Store reminder sent or not

3. Table Name: contactmessages

Sr_No	Field Name	Data Type	Description
1	id(PK)	Int(series)	Store contact id
2	name	varchar(255)	Store name
3	phone	varchar(20)	Store phone
4	email	varchar(255)	Store email
5	status	varchar(50)	Store contact status
6	created_at	timestamp	Store creation time

4. Table Name: Documents

Sr_No	Field Name	Data Type	Description
1	id(PK)	Int(serial)	Store document id
2	report_id(FK)	varchar(255)	Store report id
3	name	varchar(255)	Store document name
5	url	text	Store document url
6	type	varchar(50)	Store document type
7	created_at	timestamp	Store creation time

5. Table Name: Faq

Sr_No	Field Name	Data Type	Description
1	id(PK)	int(serial)	Store faq id
2	question	text	Store faq question
3	answer	text	Store faq answer
4	display order	integer	Store faq display id

6. Table Name: Medicines

Sr_No	Field Name	Data Type	Description
1	id(PK)	Int(serial)	Store medicine id
2	report_id	integer	Store report id
3	name	varchar(255)	Store medicine name
4	dosage	varchar(255)	Store medicine dosage
5	created_at	timestamp	Store creation time

7. Table Name: Reports

Sr_No	Field Name	Data Type	Description
1	id(PK)	series	Store report id
2	appointment_id(FK)	varchar(255)	Store appointment id
3	diagnosis	text	Store diagnosis
4	observations	text	Store observations
5	treatment	text	Store treatment
6	next_visit	date	Store next visit date
7	created_at	timestamp	Store creation time

8. Table Name: Reviews

Sr_No	Field Name	Data Type	Description
1	id(PK)	serial	Store review id
2	Patient_name	varchar (100)	Store patient name
3	Service	varchar (100)	Store service name
4	rating	integer	Store rating
5	comment	text	Store comments

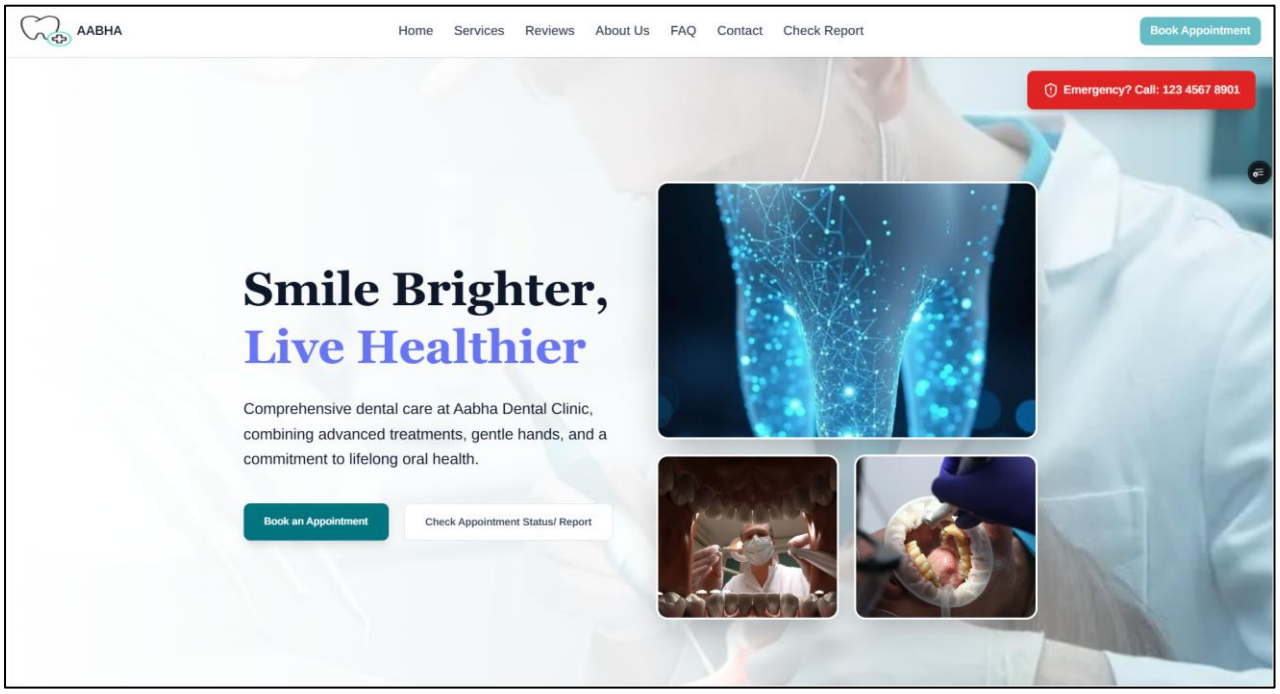
9. Table Name: Services

Sr_No	Field Name	Data Type	Description
1	id(PK)	serial	Store service id
2	name	Varchar(255)	Store service name
3	category	Varchar(255)	Store service category
4	description	text	Store service description
5	Image_url	text	Store service url
6	Is_active	boolean	Store service is active or not

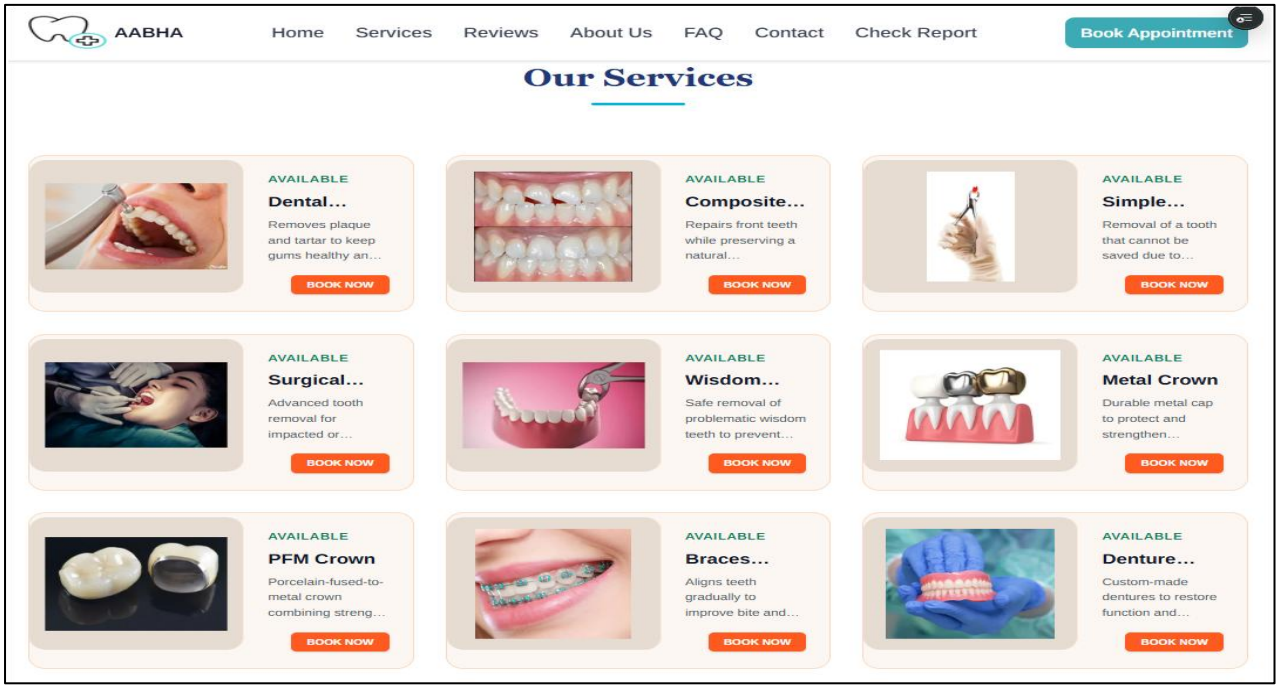
4.2 Input Design:

A) Patient:

1. Home Page:

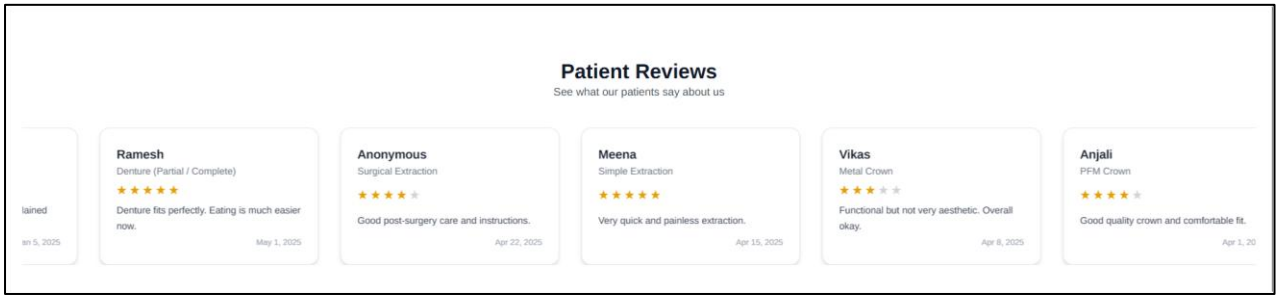


2. Services Section:

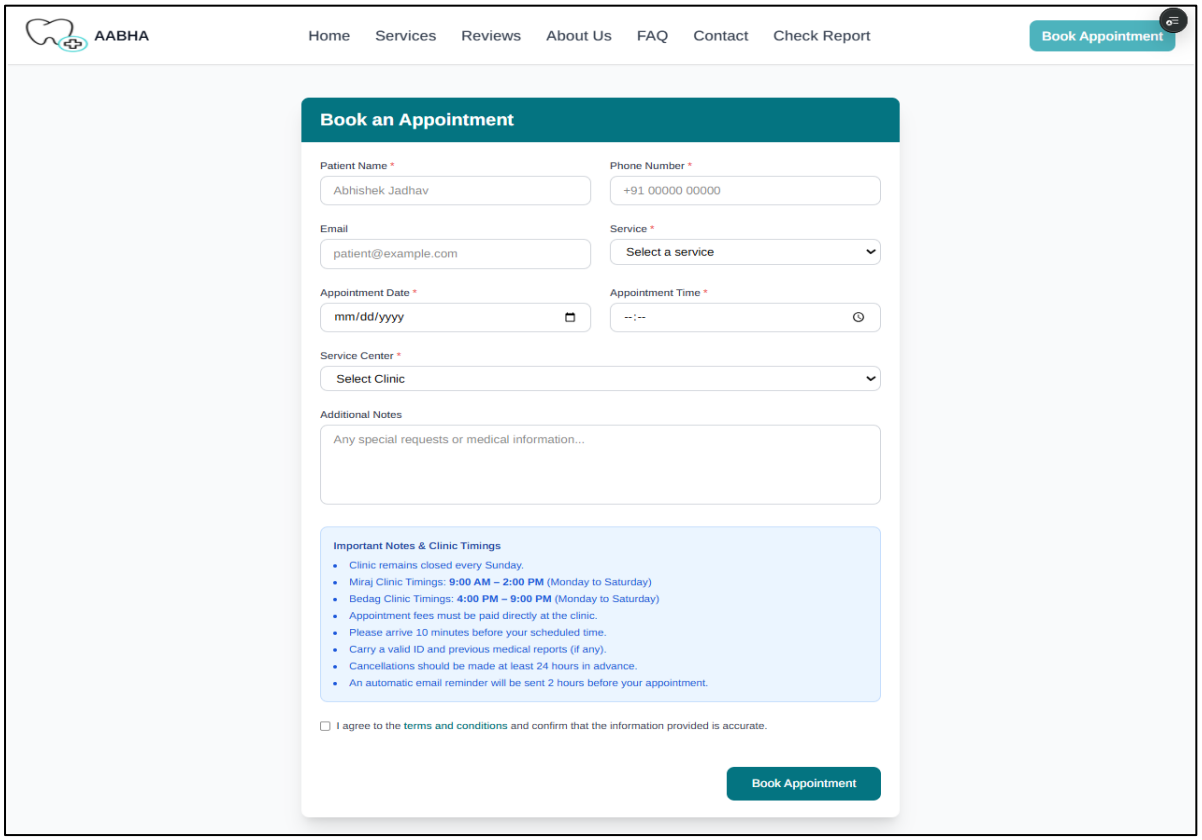


3. About Us Section :

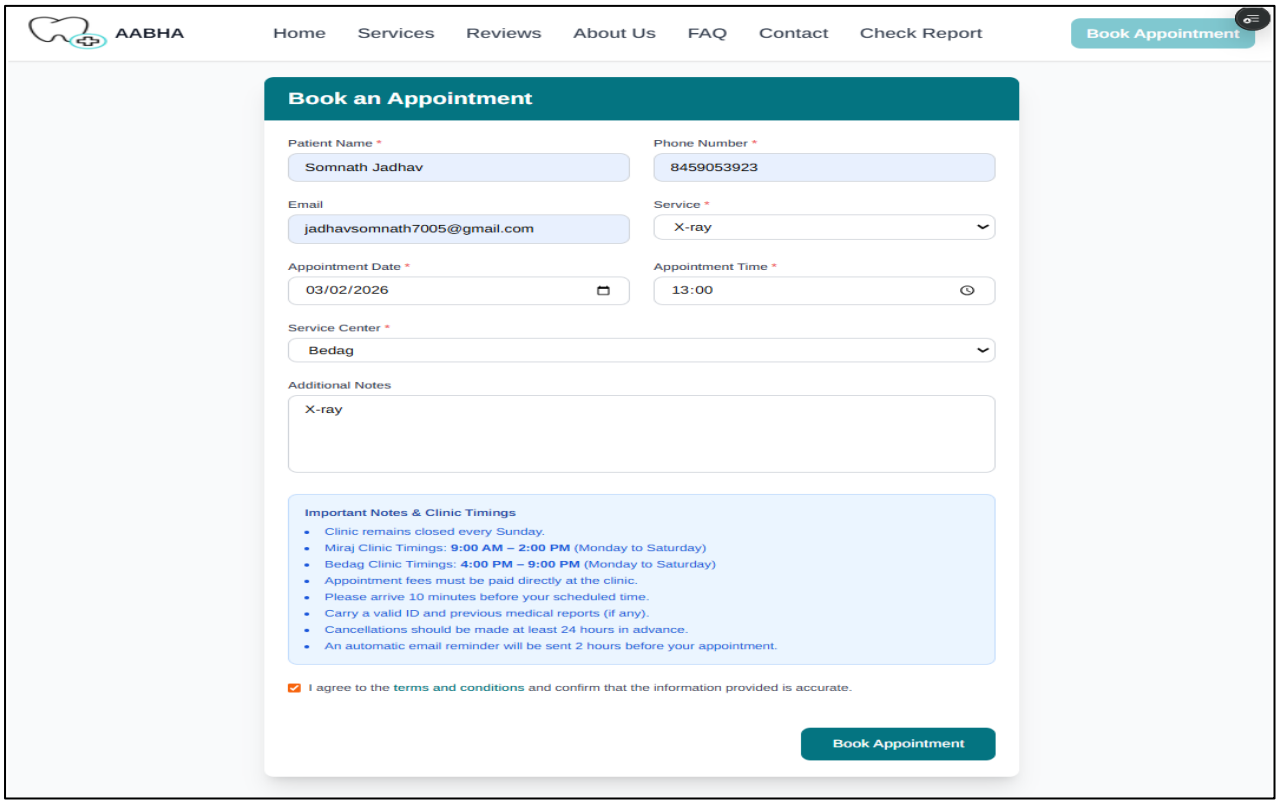
4. Reviews Section:



5. Appointment Booking Page (without input data):



With Input Data –

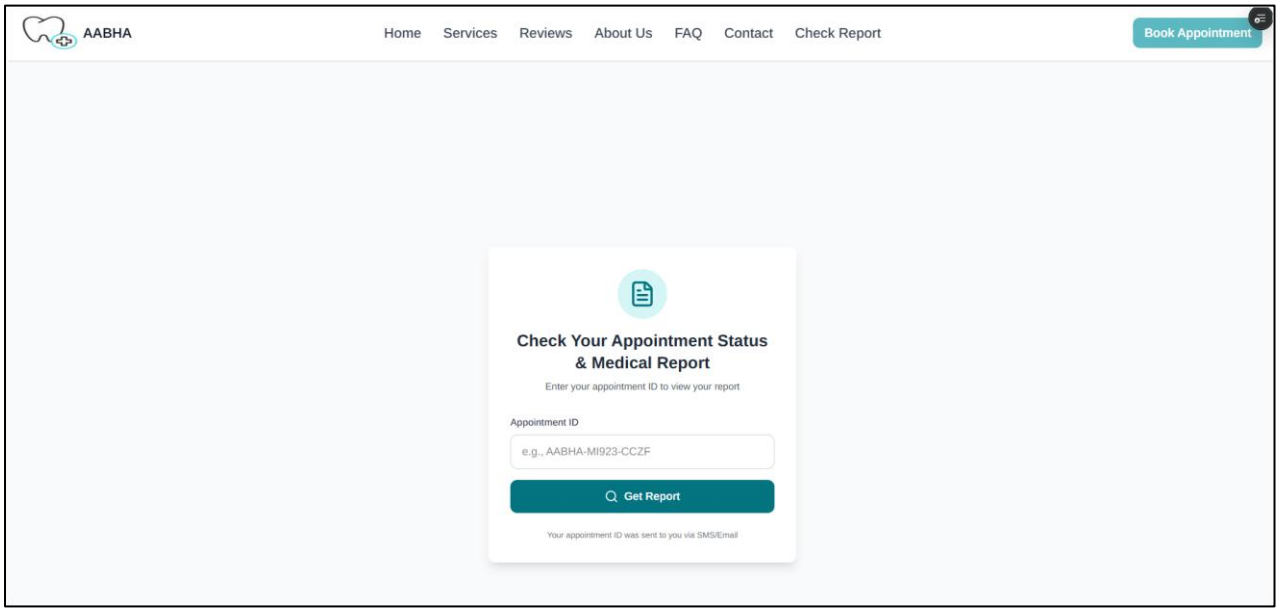


Source Code:

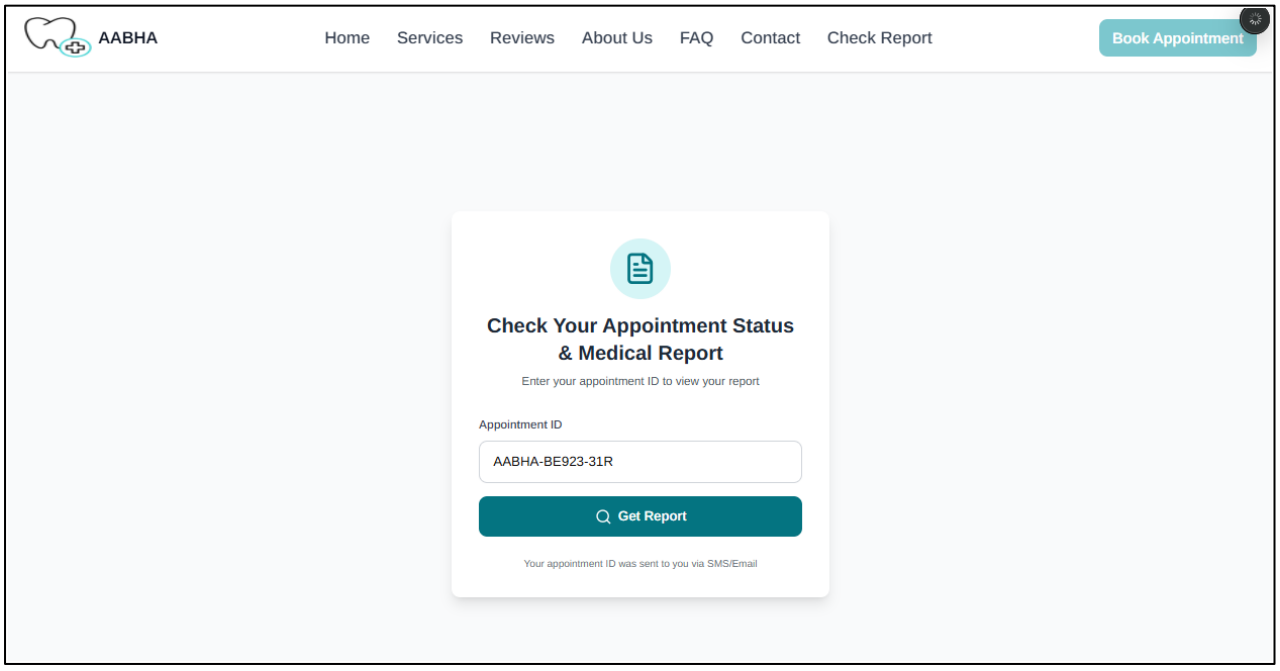
```
import db from "@/lib/db";
import { NextResponse } from "next/server";
export async function POST(request) {
  try {
    const {
      patient_name, phone, email, service, appointment_date,
appointment_time, clinic, notes, } = await request.json();
    if (
      !patient_name ||phone ||!service ||!appointment_date
||!appointment_time ||!clinic
    ) {
      return Response.json(
        { error: "Missing required fields" },
        { status: 400 },
      );
    }
    const clean = new Date(appointment_date).toLocaleString("en-
IN", {
      day: "2-digit",month: "short",
      year: "numeric",
      hour: "2-digit",
      minute: "2-digit",
      hour12: true,
    });
    const rand = Math.random().toString(36).slice(2,
6).toUpperCase();

    const uniqueId =
      "AABHA-" +
      clinic.slice(0, 2).toUpperCase() +
      phone.slice(-3) +
      "-" +
      rand;
    const now = new Date();
    const createdAt = now.toISOString();

    const result = await db.query(
      "INSERT INTO appointments(name, phone, email, service_name,
```

```
preferred_date, preferred_time, status, notes, created_at,
appointment_id, at) VALUES($1, $2, $3, $4, $5, $6, $7, $8, $9,
$10, $11) RETURNING *",
      [
        patient_name,phone, email || null, clean,
appointment_time,"Pending",notes || null, createdAt, uniqueId,
clinic,],
    );
    return NextResponse.json(
      {
        success: true,
        appointment: result.rows[0],
        message: "Appointment booked successfully",
      },
      { status: 201 },
    );
  } catch (error) {
    console.error("Error creating appointment:", error);
    return NextResponse.json(
      { error: "Failed to create appointment" },
      { status: 500 },
    );
  }
}
```

6. Appointment Status/report Checking (without data) –



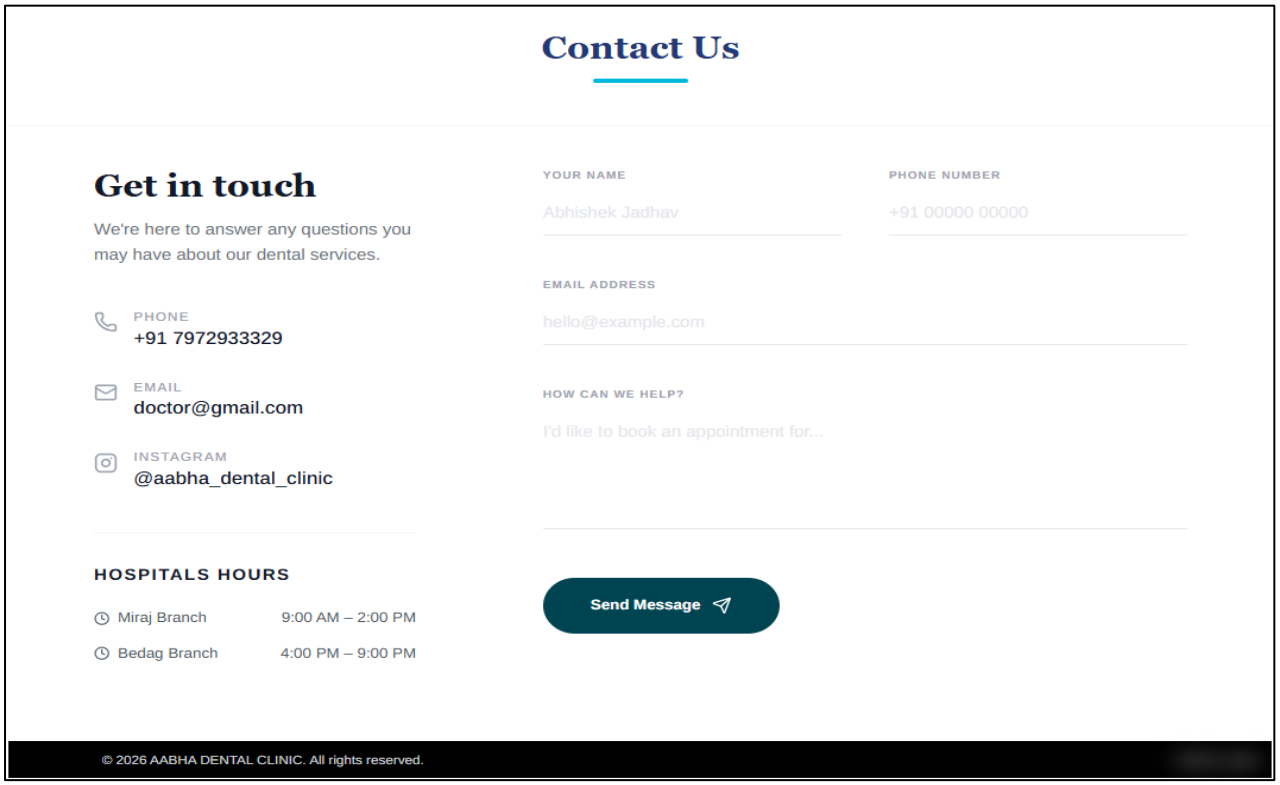
With Data:



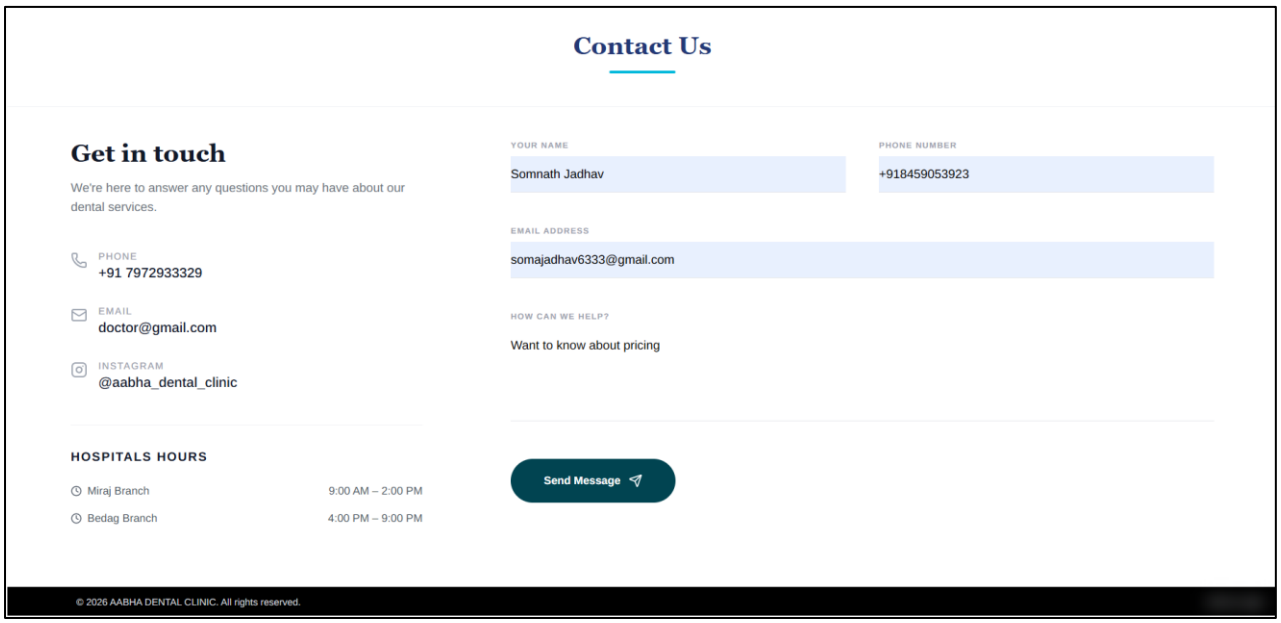
Source code :

```
import db from "@/lib/db";
import { NextResponse } from "next/server";
export async function GET(request) {
  try {
    const result = await db.query(
      "SELECT * FROM appointments ORDER BY created_at DESC",
    );
    return NextResponse.json({ success: true, data: result.rows
});
  } catch (error) {
    console.error("Error fetching appointments:", error);
    return NextResponse.json(
      { success: false, error: "Failed to fetch appointments" },
      { status: 500 },
    );
  }
}
```

7. Contact Us (without Data):



With data:

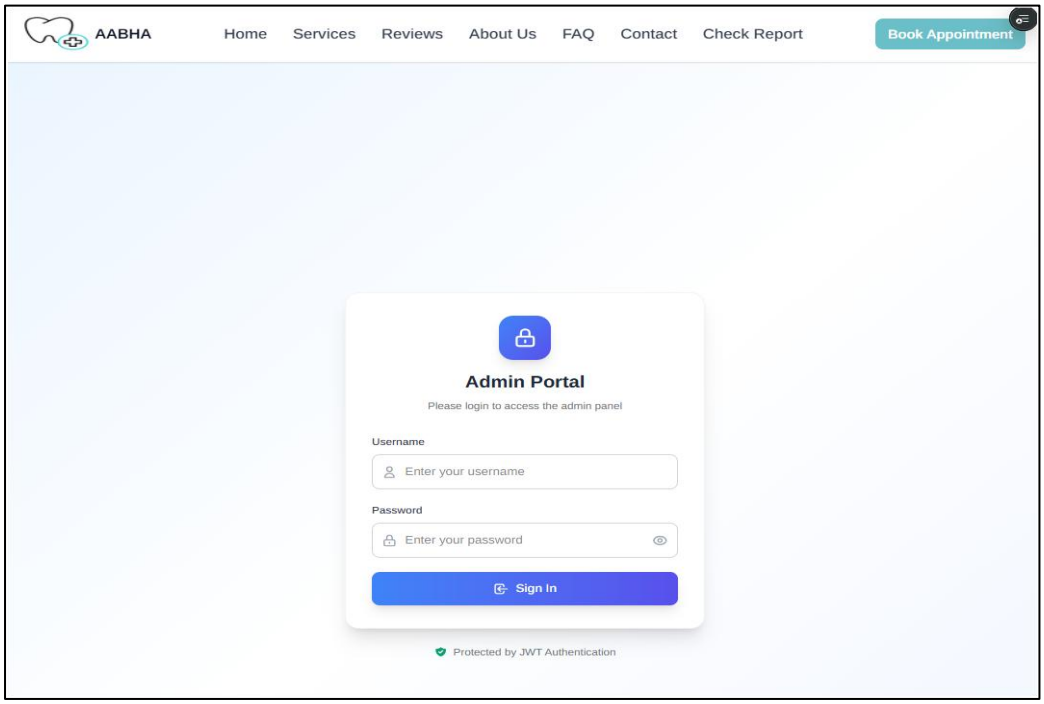


Source code:

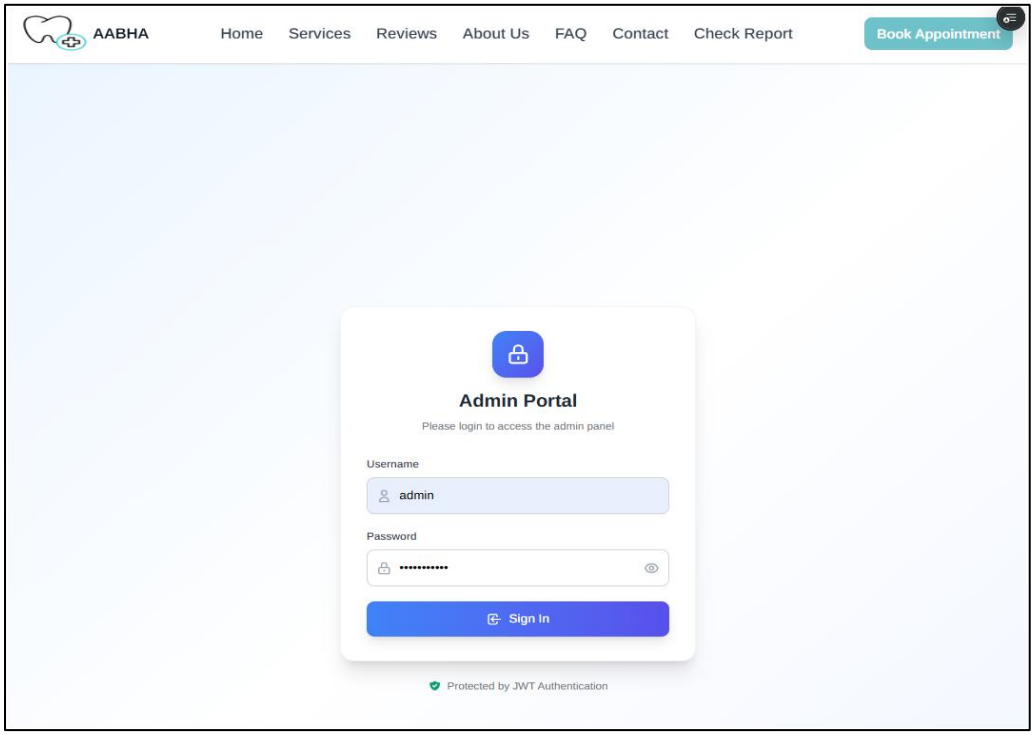
```
import db from "@/lib/db";
import { NextResponse } from "next/server";

export async function POST(request) {
  const { name, phone, email, message } = await request.json();
  const status = "pending";
  const now = new Date();
  const createdAt = now.toISOString();
  try {
    const cnMessage = await db.query(
      "INSERT INTO contactmessages(name, phone, email, status,
created_at, messages) VALUES($1,$2,$3,$4,$5,$6)",
      [name, phone, email, status, createdAt, message],
    );

    if (cnMessage) {
      return NextResponse.json({
        status: "Success",
        message: "We got Your Message",
      });
    }
  } catch (error) {
    console.log(error);
  }
  return NextResponse.json({
    status: "Success",
    message: "We got Your Message",
  });
}
```

B) Admin –

Admin Login Page (without Data)



With data -

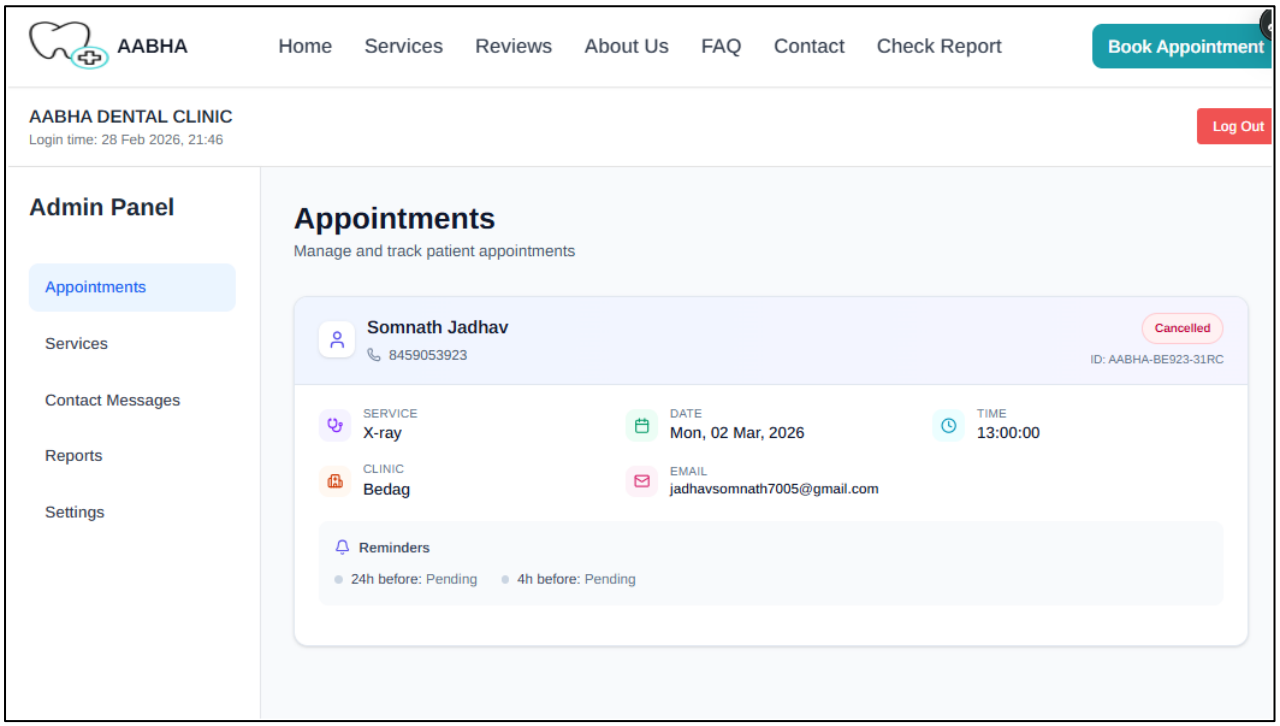


Source code:

```
import { NextResponse } from "next/server";
import jwt from "jsonwebtoken";
const JWT_SECRET =process.env.JWT_SECRET || "your-secret-key-
change-in-production";
if (!process.env.JWT_SECRET && process.env.NODE_ENV ===
"production") {
  throw new Error("JWT_SECRET is not defined in production
environment!");
}
export async function GET(request) {
  try {
    const authHeader = request.headers.get("authorization");
    if (!authHeader || !authHeader.startsWith("Bearer ")) {
      return NextResponse.json(
        { valid: false, message: "No token provided" },
        { status: 401 },
      );
    }
    const token = authHeader.substring(7);
    try {
      const decoded = jwt.verify(token, JWT_SECRET);
      return NextResponse.json(
        {
          valid: true,
          user: {
            userId: decoded.userId,
            username: decoded.username,
            role: decoded.role,
          },
          message: "Token is valid",
        },
        { status: 200 },
      );
    } catch (jwtError) {
      console.error("JWT verification failed:",
jwtError.message);
      let errorMessage = "Invalid token";
```

```
      if (jwtError.name === "TokenExpiredError") {
        errorMessage = "Token has expired";
      } else if (jwtError.name === "JsonWebTokenError") {
        errorMessage = "Invalid token signature";
      }
      return NextResponse.json(
        { valid: false, message: errorMessage },
        { status: 401 },
      );
    }
  } catch (error) {
    console.error("Verification error:", error);
    return NextResponse.json(
      { valid: false, message: "An error occurred during
verification" },
      { status: 500 },
    );
  }
}
```

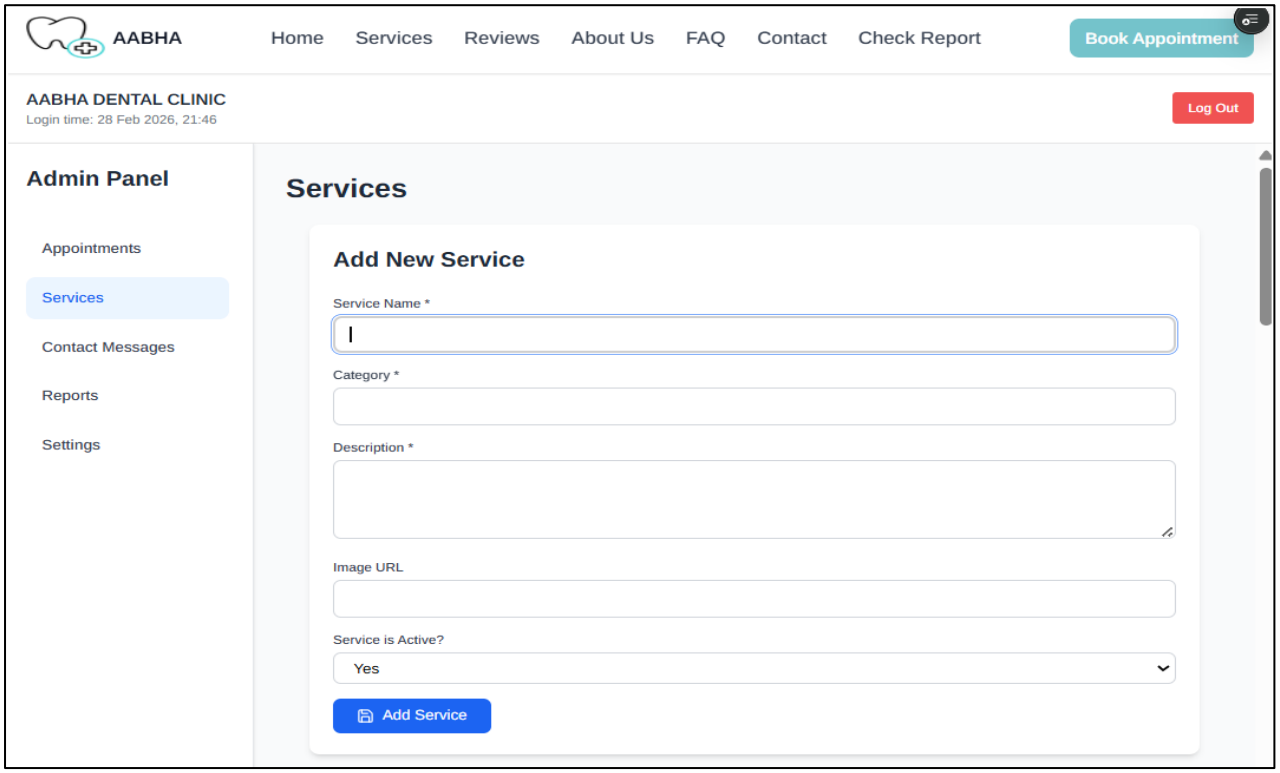
Appointments View:



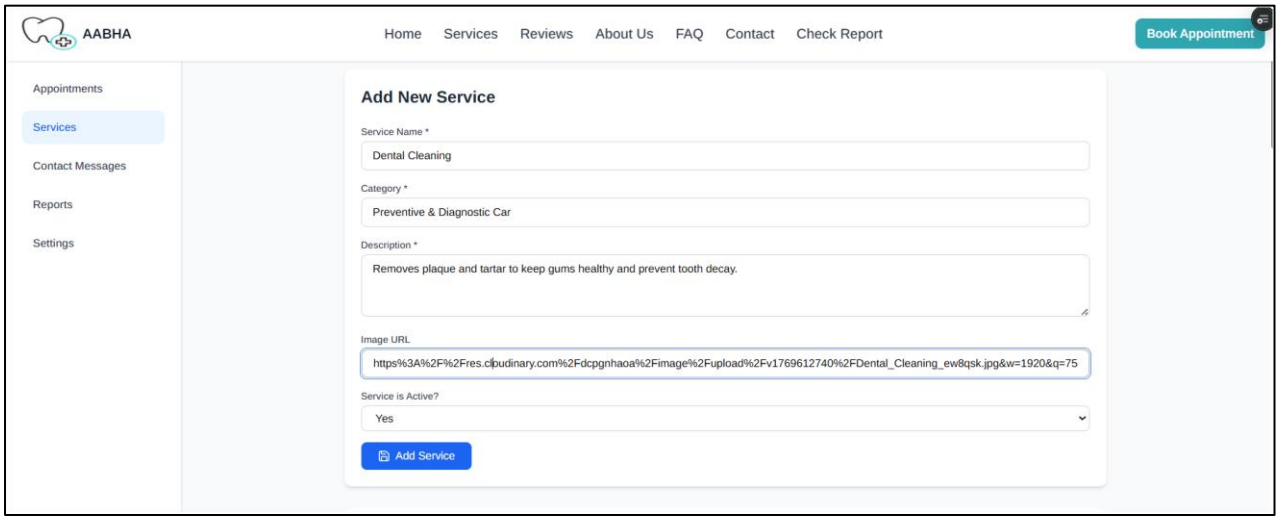
Source code:

```
import db from "@/lib/db";
import { NextResponse } from "next/server";
export async function GET(request) {
  try {
    const result = await db.query("SELECT * FROM appointments
ORDER BY created_at DESC", );
  return NextResponse.json({ success: true, data: result.rows });
  } catch (error) {
    console.error("Error fetching appointments:", error);
    return NextResponse.json(
      { success: false, error: "Failed to fetch appointments" },
      { status: 500 },
    );
  }
}
```

Add/Delete Services (without data) –



Add/Delete Services (with data) –

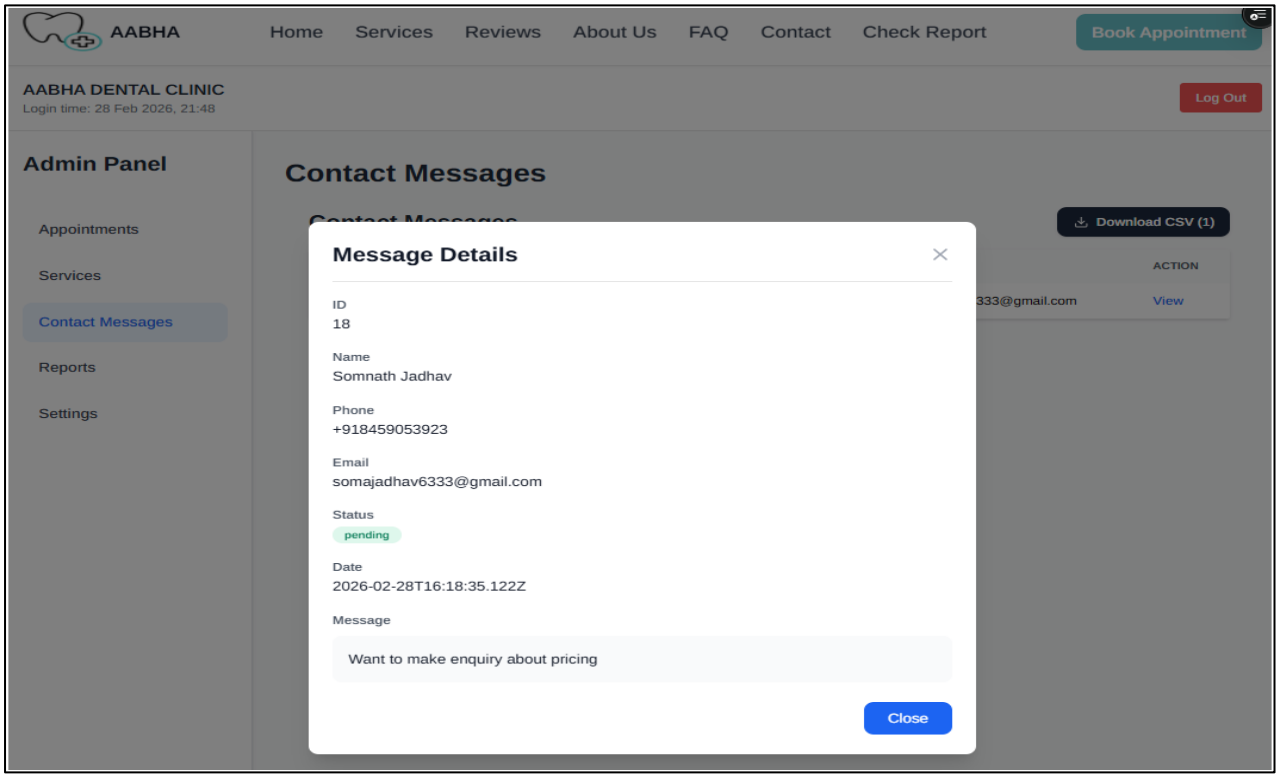


Source Code:

```
import db from "@/lib/db";
import { NextResponse } from "next/server";
export async function GET() {
  const services = (await db.query("SELECT * FROM
services")).rows;
  const categories = (await db.query("SELECT DISTINCT category
FROM services")) .rows;
  return NextResponse.json(
    {
      services: services,
      categories: categories,
    },
    { status: 200 },
  );
}
export async function POST(request) {
  const { name, category, description, image_url, is_active,
display_order } =
    await request.json();
  const result = await db.query(
    "INSERT INTO services(name, category, description, image_url,
is_active, display_order) VALUES($1, $2, $3, $4, $5, $6)
RETURNING *",
```

```
    [name, category, description, image_url, is_active,
display_order],
  );
  if (result.rowCount === 0) {
    throw new Error("Error while creating new service");
  }
  return NextResponse.json(result.rows[0], { status: 200 });
}
export async function PUT(request) {
  const { id, name, category, description, image_url, is_active }
=await request.json();
  const result = await db.query(
    "UPDATE services SET name = $1, category = $2, description =
$3, image_url = $4, is_active = $5 WHERE id = $6 RETURNING *",
    [name, category, description, image_url, is_active, id],
  );
  if (result.rowCount === 0) {
    throw new Error("Error while updating");
  }
  return NextResponse.json(result.rows[0], { status: 200 });
}
export async function DELETE(request) {
  const { id } = await request.json();
  const result = await db.query("DELETE FROM services WHERE id =
$1 RETURNING *",[id],
  );
  if (result.rowCount === 0) {
    throw new Error("Error while deleting service");
  }
  return NextResponse.json(
    {
      message: "Service deleted successfully",
      deleted: result.rows[0],
    },
    { status: 200 },
  );
}
```

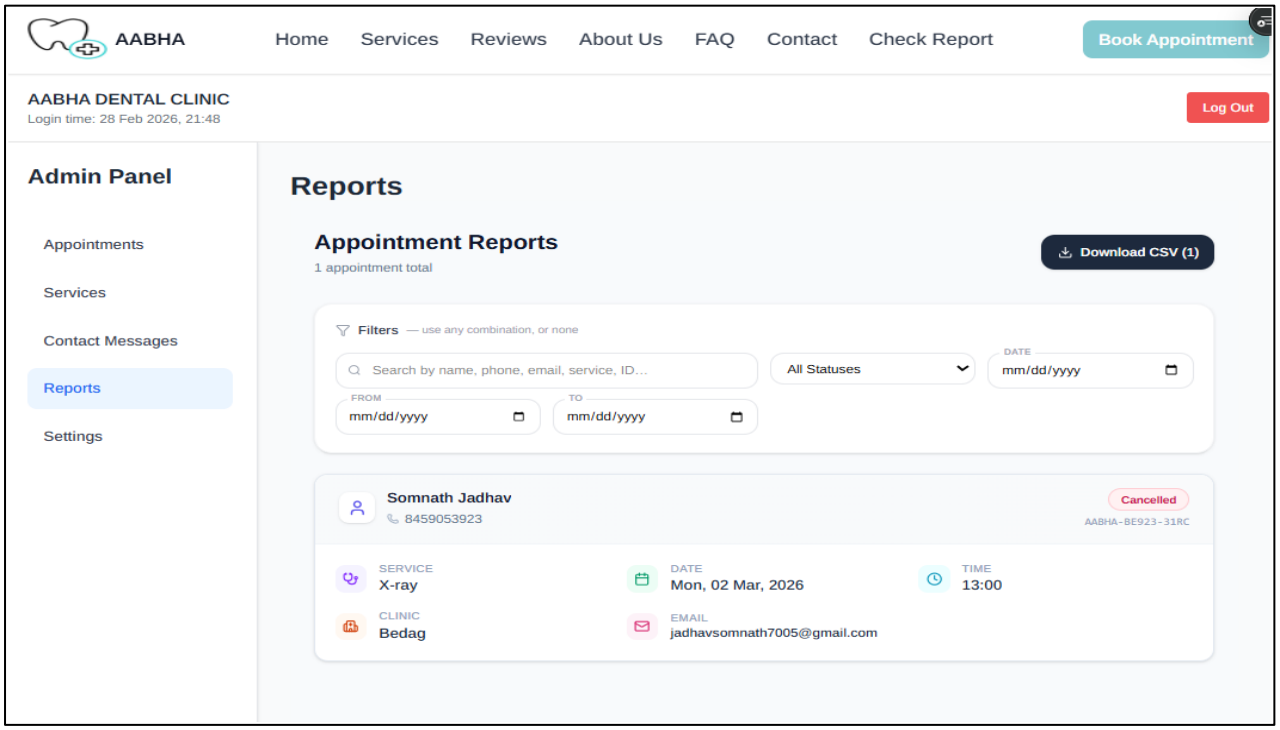
Contact Messages:



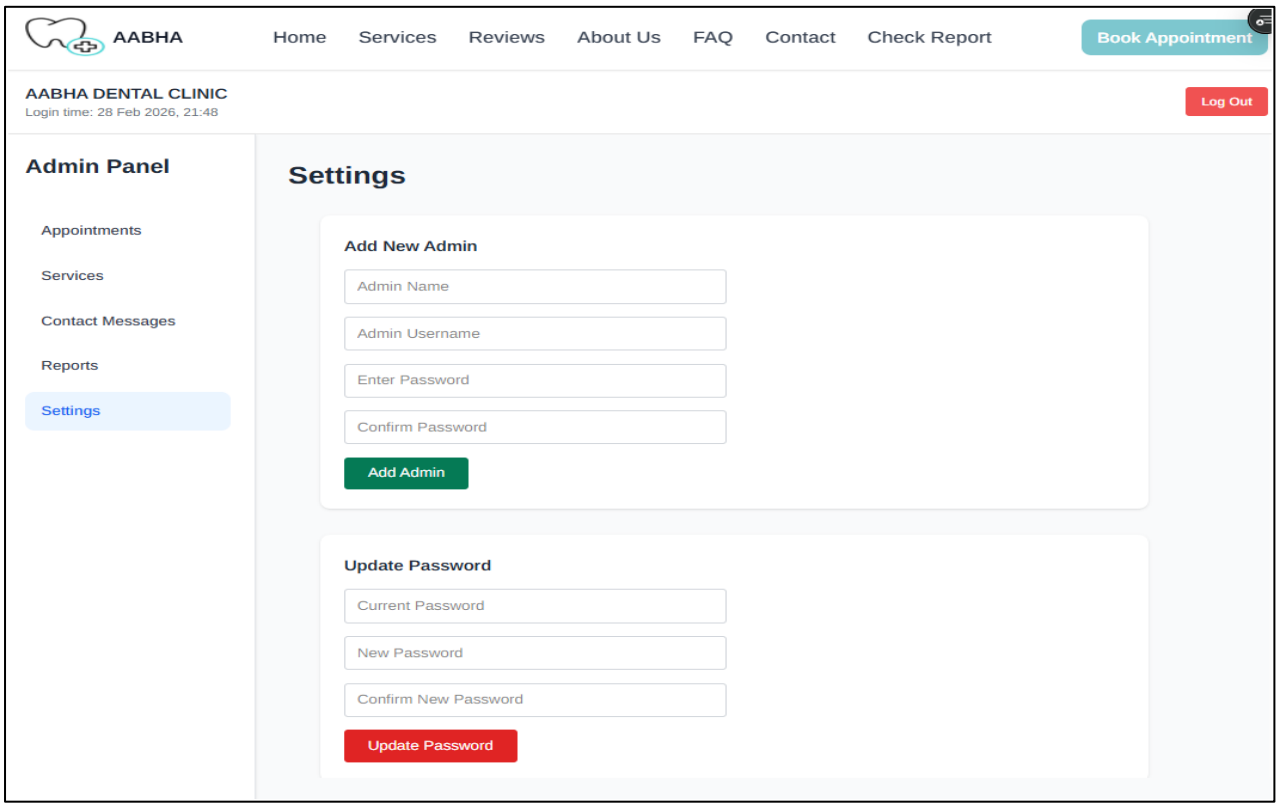
Source Code :

```
import db from "@/lib/db";
import { NextResponse } from "next/server";

export async function GET() {
  const data = await db.query("SELECT * FROM contactmessages");
  return NextResponse.json({
    data: data.rows,
    status: "Success",
  });
}
```

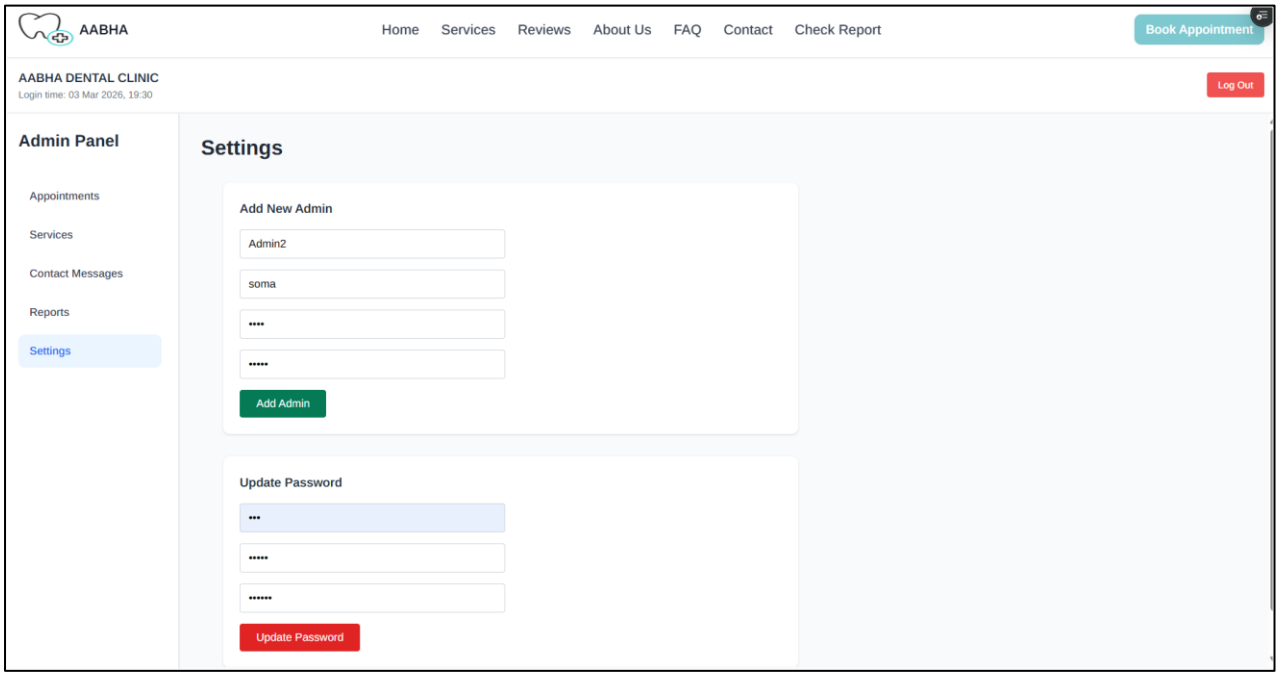
Reports view:



Settings Page (without Data):



With data:

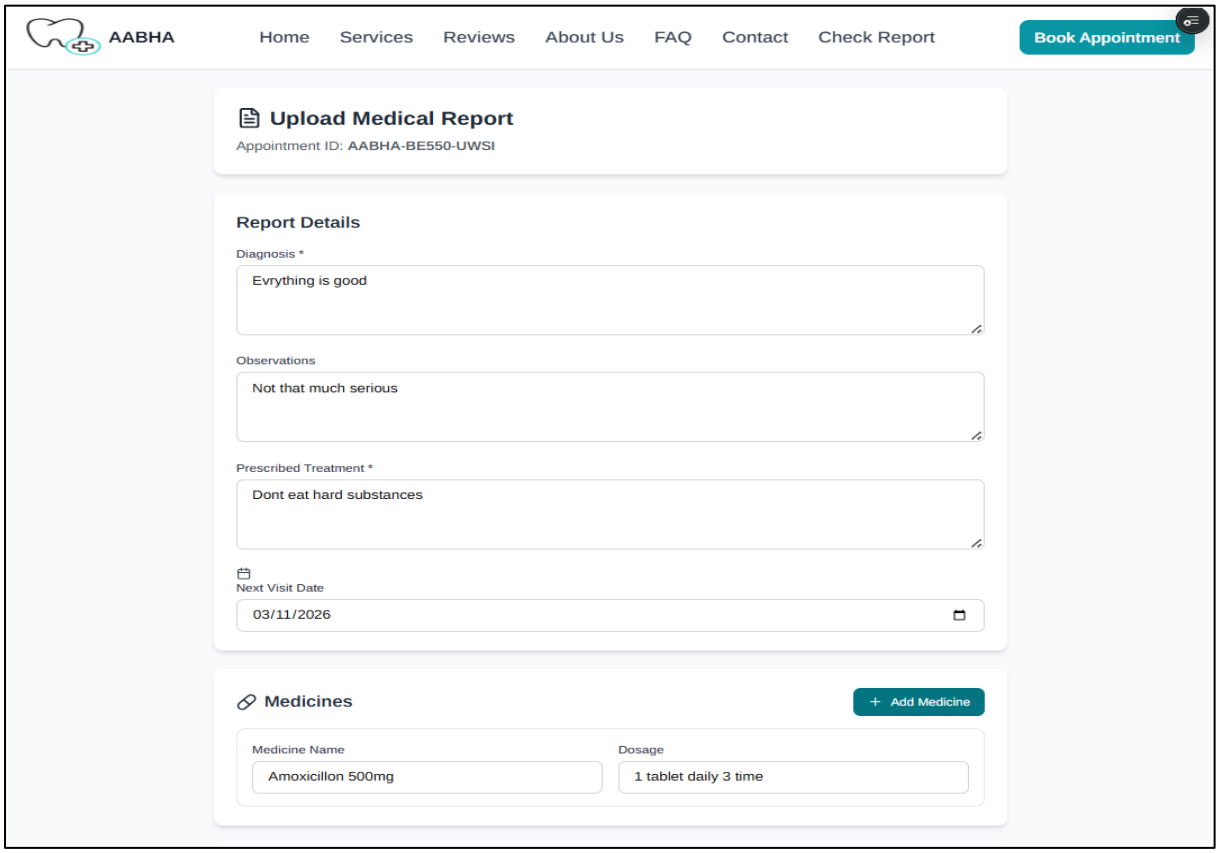


Source Code :

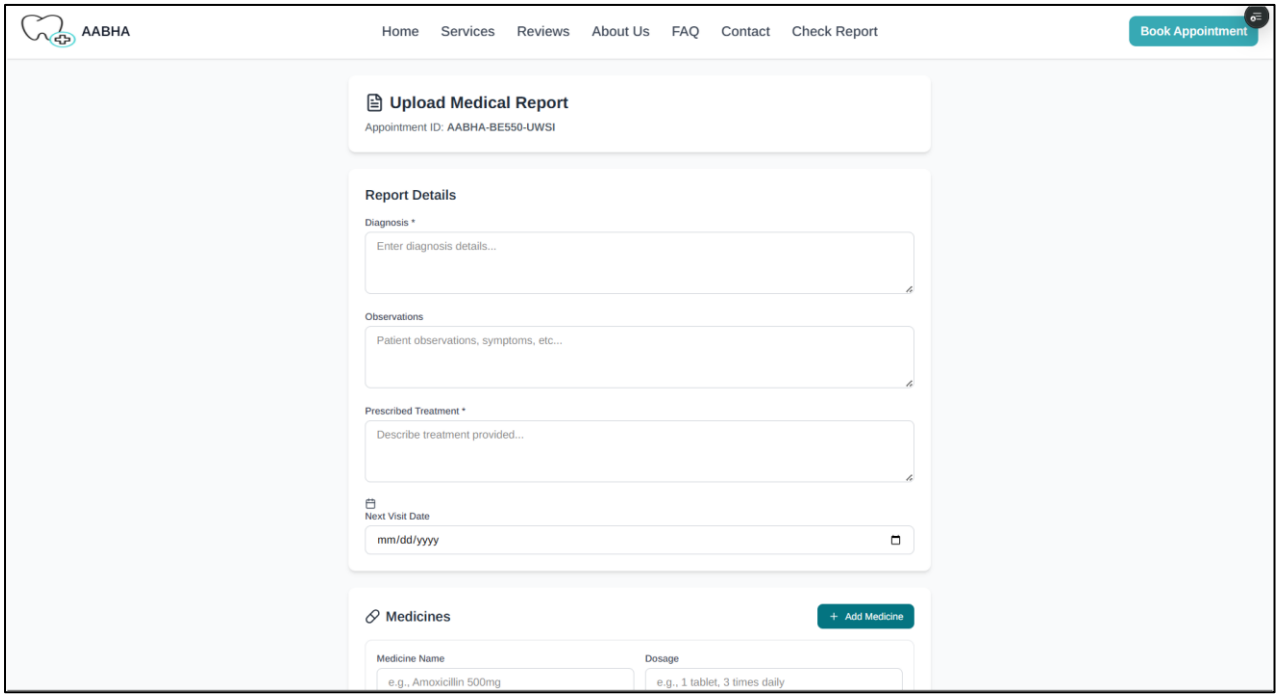
```
import db from "@/lib/db";
import { NextResponse } from "next/server";
import bcrypt from "bcrypt";

export async function POST(request) {
  try {
    const { name, username, password } = await request.json();
    if (!name || !username || !password) {
      return NextResponse.json(
        { error: "Name, username and password are required" },
        { status: 400 },
      );
    }
    if (password.length < 6) {
      return NextResponse.json(
        { error: "Password must be at least 6 characters" },
        { status: 400 },
      );
    }
```

```
    const existing = await db.query(
      "SELECT id FROM admin WHERE username = $1",
      [username],
    );
    if (existing.rows.length > 0) {
      return NextResponse.json(
        { error: "An admin with this username already exists" },
        { status: 409 },
      );
    }
    const hashedPassword = await bcrypt.hash(password, 10);
    const result = await db.query(
      "INSERT INTO admin (name, username, password, created_at)
VALUES ($1, $2, $3, $4) RETURNING id, name, username,
created_at",
      [name, username, hashedPassword, new Date().toISOString()],
    );
    return NextResponse.json(
      {
        success: true,
        message: "Admin added successfully",
        admin: result.rows[0],
      },
      { status: 201 },
    );
  } catch (error) {
    console.error("Error adding admin:", error);
    return NextResponse.json({ error: "Failed to add admin" }, {
status: 500 });
  }
}
```

Reports Upload (Without Data):



With Data:



Source Codes:

```
export async function POST(request) {
  try {
    const {appointment_id, diagnosis, observations, treatment, next_visit, medicines, documents, } = await request.json();
    if (!appointment_id || !diagnosis || !treatment) {
      return NextResponse.json(
        { error: "Appointment ID, Diagnosis, and Treatment are required" },
        { status: 400 }, );}
    await db.query("BEGIN");
    try {
      const reportResult = await db.query(
        "INSERT INTO reports(appointment_id, diagnosis, observations, treatment, next_visit) VALUES($1, $2, $3, $4, $5) RETURNING id",
      const reportId = reportResult.rows[0].id;
            if (medicines && medicines.length > 0) {
        for (const medicine of medicines) {
          if (medicine.name && medicine.dosage) {
            await db.query(
              "INSERT INTO medicines(report_id, name, dosage) VALUES($1, $2, $3)",
              [reportId, medicine.name, medicine.dosage],
            );
          }
        }
      }
```

```
      if (documents && documents.length > 0) {
        for (const document of documents) {
          if (document.name && document.url) {
            await db.query(
              "INSERT INTO documents(report_id, name, url, type)
VALUES($1, $2, $3, $4)",
              [reportId,        document.name,        document.url,
document.type || "image"],
            ); }}}
      await db.query("COMMIT");
return NextResponse.json(
        {
          success: true,
          message: "Report uploaded successfully",
          reportId: reportId,
        },
        { status: 200 },
      );
  } catch (error) {
    console.error("Error while uploading report:", error);
    return NextResponse.json(
      {error: "upload failed",
      },
      {status: 500 },
    );
  }
}
```

4.3Output Design:

A. Reports:

1. Appointments Report:

	A	B	C	D	E	F	G	H	I
1	Appointment ID	Patient Name	Email	Phone	Service	Date	Time	Clinic	Status
2	AABHA-BE550-UWSI	Vaibhav Mohite	somajadhav6333@gmail.com	7050150550	Surgical Extraction	2026-03-03T00:00:00.0	13:17:00	Bedag	Completed
3	AABHA-BE923-31RC	Somnath Jadhav	jadhavsomnath7005@gmail.com	8459053923	X-ray	2026-03-02T00:00:00.0	13:00:00	Bedag	Cancelled
4									

2. Services Report:

K22

	A	B	C	D	E	F	G	H
1	ID	Name	Category	Description	Image URL	Active	Display Order	
2	3	Dental Cleaning (Scaling)	Preventive & Diagnostic Car	Removes plaque and tartar	https://res	Yes	3	
3	5	Composite Restoration (Anteri	Tooth Restoration	Repairs front teeth while pr	https://res	Yes	5	
4	6	Simple Extraction	Tooth Extraction	Removal of a tooth that car	https://res	Yes	6	
5	7	Surgical Extraction	Tooth Extraction	Advanced tooth removal for	https://res	Yes	7	
6	8	Wisdom Tooth Extraction	Tooth Extraction	Safe removal of problematic	https://res	Yes	8	
7	10	Metal Crown	Crowns & Caps	Durable metal cap to protec	https://res	Yes	11	
8	11	PFM Crown	Crowns & Caps	Porcelain-fused-to-metal cr	https://res	Yes	12	
9	13	Braces (Orthodontic Treatmen	Orthodontics	Aligns teeth gradually to imp	https://res	Yes	14	
10	15	Denture (Partial / Complete)	Dentures	Custom-made dentures to r	https://res	Yes	16	
11	12	Zirconia Crown	Crowns & Caps	High-strength, natural-lookin	https://res	Yes	13	
12	1	Dental Check-up	Preventive & Diagnostic Car	Routine examination to asse	https://res	Yes	1	
13	9	Root Canal Treatment	Root Canal Treatment	Treats infected tooth pulp t	https://res	Yes	9	
14	2	X-ray	Preventive & Diagnostic Car	Dental imaging to identify hi	https://res	Yes	2	
15	4	Composite Restoration (Poster	Tooth Restoration	Restores damaged back tee	https://res	No	4	
16	14	Teeth Whitening	Cosmetic Dentistry	Brightens teeth safely for a	https://res	Yes	15	

3. Contact Messages Report:

K22

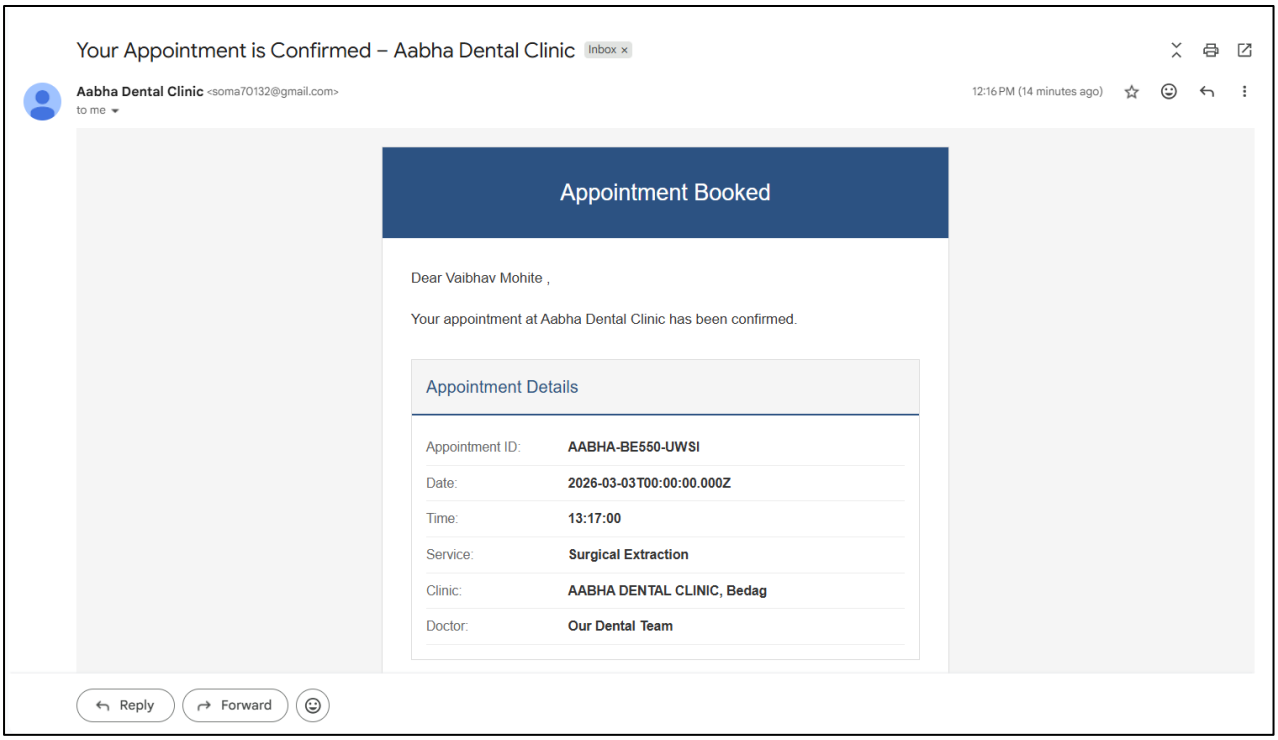
	A	B	C	D	E	F	G	H	I	J
1	ID	Name	Phone	Email	Status	Date	Message			
2	18	Somnath Jadhav	9.185E+11	somajadhav6333@	pending	2026-02-28	Want to make enquiry about pricing			
3										
4										

Source Code:

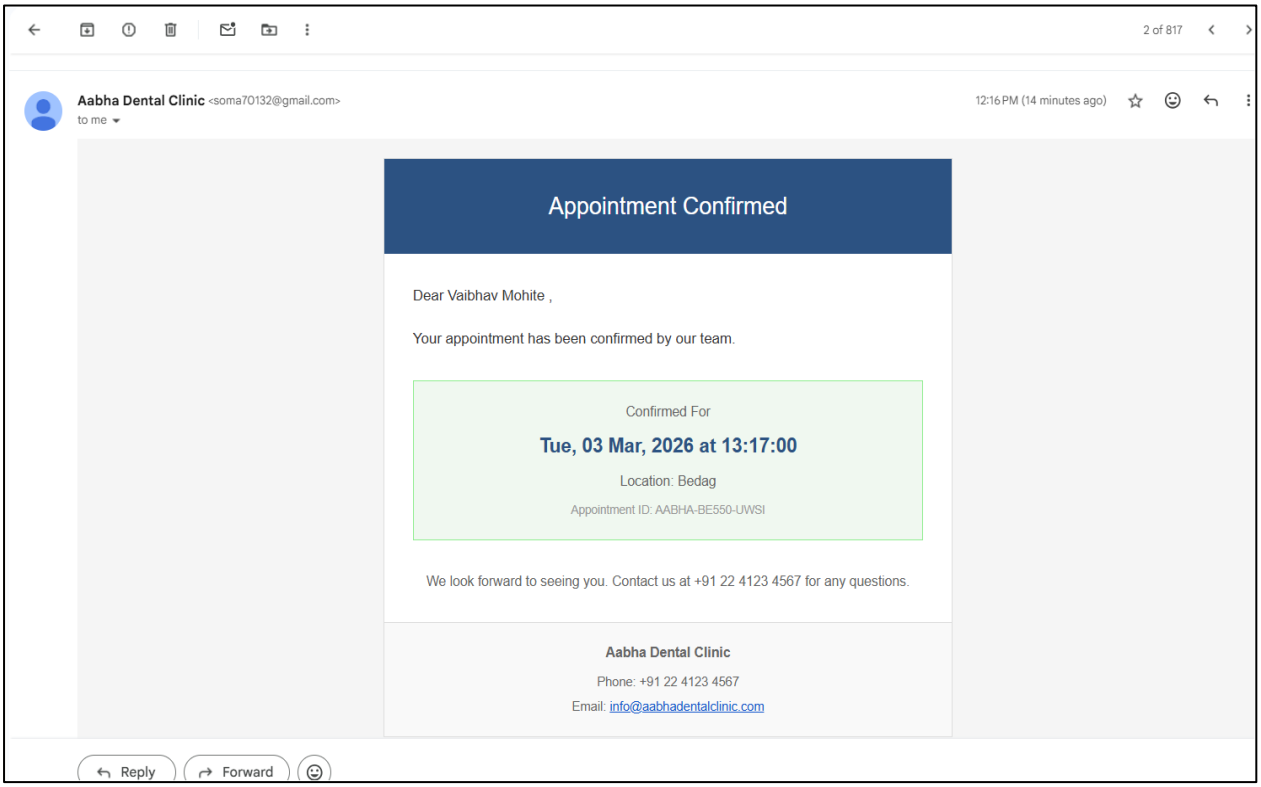
```
const downloadCSV = (data) => {
  if (!data.length) return;
  const headers = ["ID", "Name", "Phone", "Email", "Status", "Date", "Message"];
  const lines = [headers.join(","),...data.map((msg) =>
      [msg.id,`"${msg.name}"`,.phone, msg.email || "",msg.status,
        msg.created_at,
        `"${msg.messages?.replace(/"/g, '""')}"`,
      ].join(","),
    ),
  ];
  const blob = new Blob([lines.join("\n")], { type: "text/csv" });
  const url = URL.createObjectURL(blob);
  const a = document.createElement("a");
  a.href = url;
  a.download = `contact-messages-${new Date().toISOString().split("T")[0]}.csv`;
  a.click();
  URL.revokeObjectURL(url);
};
```

B. Reminders:

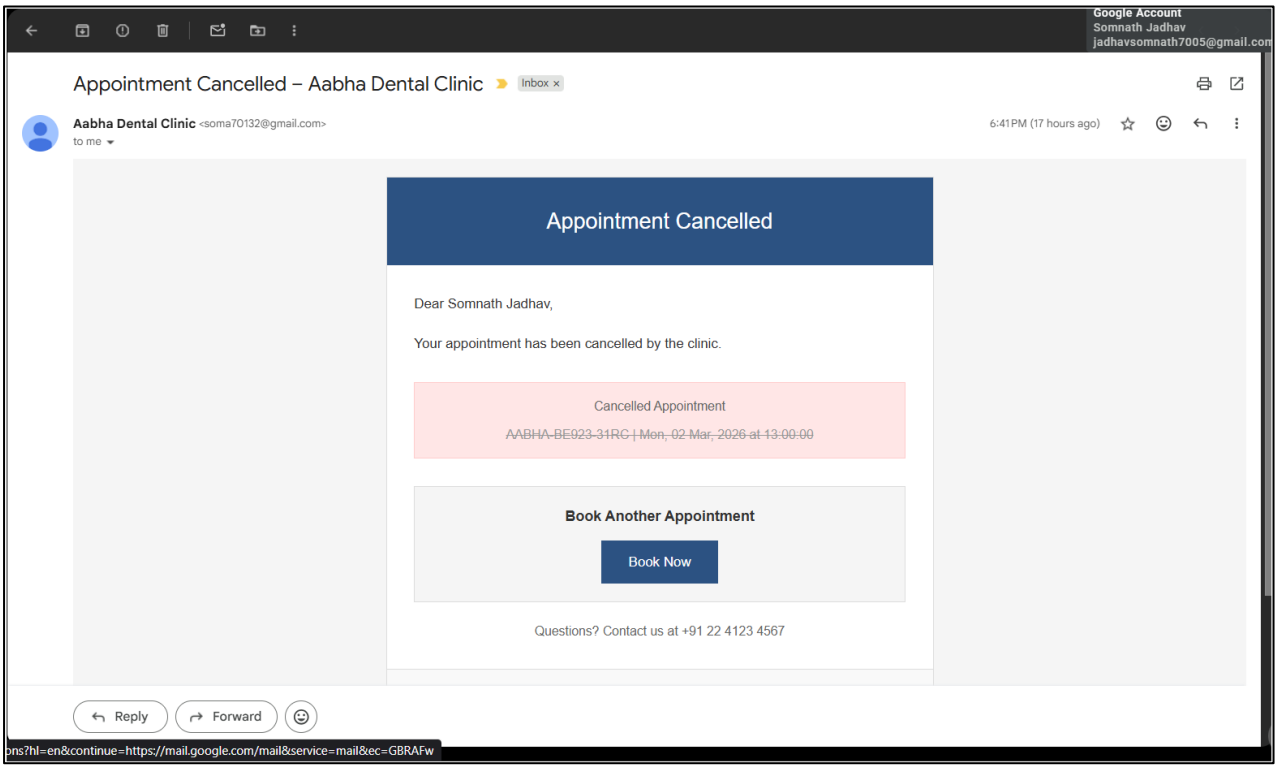
1. Appointment Booking Reminder:



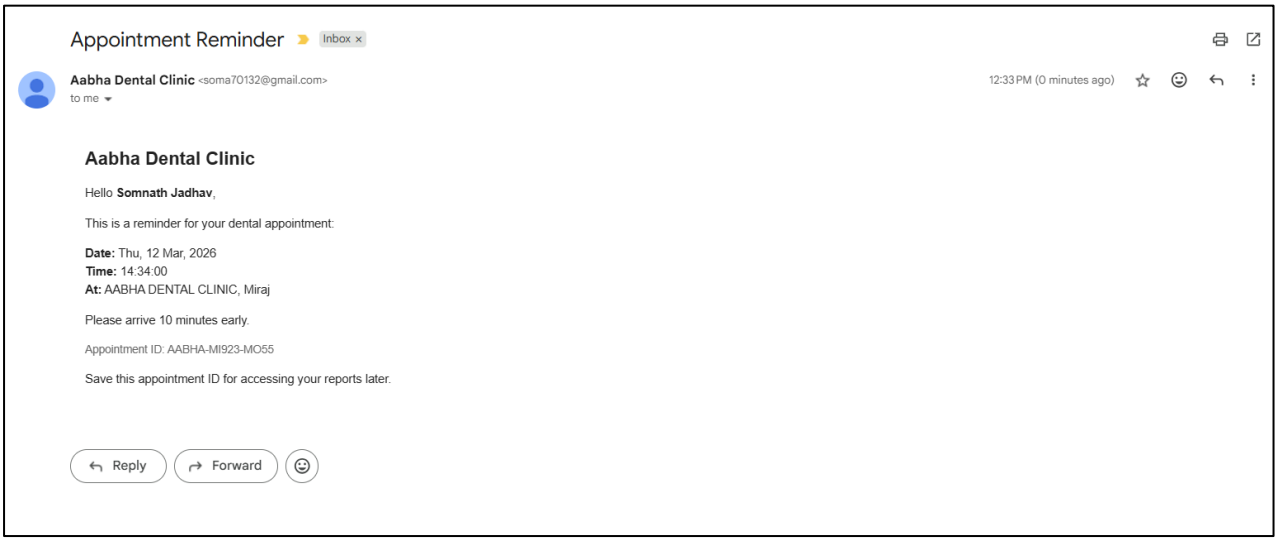
2. Appointment Confirmation reminder



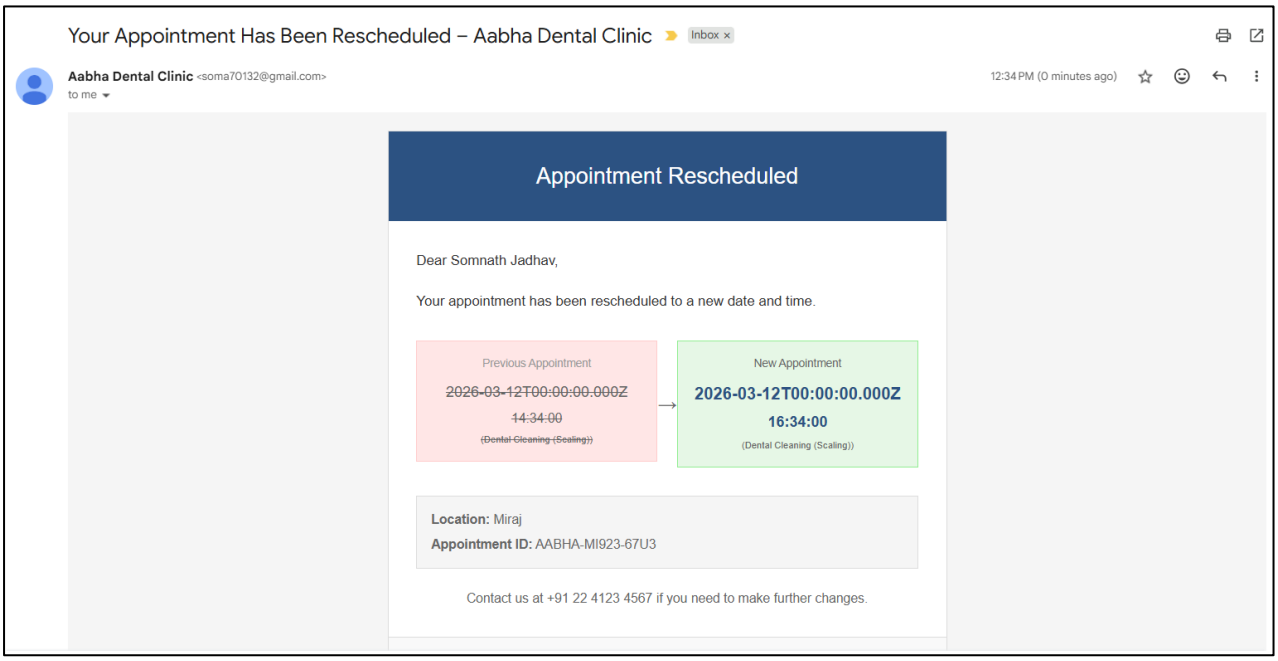
3. Appointment Cancellation Reminder:



4. Appointment Reminder:



5. Appointment Rescheduled Reminder:



Source Code:

```
import { sendEmail } from "@/lib/email";
import { NextResponse } from "next/server";
export async function POST(request) {
  try {
    const { to, subject, text, html } = await request.json();

    await sendEmail({
      to,
      subject,
      text,
      html,
    });

    return NextResponse.json(
      { success: true, message: "Reminder sent successfully" },
      { status: 200 },
    );
  } catch (error) {
    console.error("Email error:", error);
    return NextResponse.json(
      { success: false, message: "Failed to send reminder" },
      { status: 500 },
    );
  }
}
```

```
name: Appointment Reminder Cron

on:
  schedule:
    - cron: "*/10 * * * *" # every 10 minutes
  workflow_dispatch: # manual trigger button

jobs:
  trigger-reminders:
    runs-on: ubuntu-latest
    steps:
      - name: Call Vercel Cron API
        run: |
          curl       -X       GET       "https://dental-clinic-two-
omega.vercel.app/api/cron/reminders" \
          -H "Authorization: Bearer ${{ secrets.CRON_SECRET }}"
```

5.1 User Manual:

1. Introduction

The Dental Clinic Appointment System is a web-based application designed to manage appointments, patient details, and automated reminders for a dental clinic. This system helps reduce manual work and improves overall efficiency.

2. System Access

The system has two types of users:

1. Patient
2. Admin

3. Patient User Guide

3.1 Viewing Services

- Open the website.
- Navigate to the Services section.
- View the list of available dental treatments.

3.2 Booking an Appointment

1. Go to the Appointment page.
2. Fill in the required details:
    - Name
    - Contact number
    - Email
    - Preferred date
    - Preferred time
3. Click the Submit button.
4. A confirmation message will be displayed after successful booking.

3.3 Receiving Reminder Emails

- The system automatically sends:
    - One reminder 24 hours before appointment

    - One reminder 4 hours before appointment
- No action is required from the patient side.

4. Admin User Guide

4.1 Admin Login

1. Go to the Admin Login page.
2. Enter username and password.
3. Click Login.

4.2 Viewing Appointments

- After login, the dashboard displays all appointments.
- Admin can:
    - View appointment details
    - Check patient information
    - See appointment status

4.3 Filtering Appointments

- Filter by:
    - Date
    - Status (Pending, Confirmed, Completed, Cancelled)
    - Search by patient name

4.4 Updating Appointment Status

- Select an appointment.
- Change status as required.
- Save changes.

4.5 Managing Services

- Add new service
- Edit existing service
- Delete service

4.6 Viewing Contact Messages

- Admin can view messages submitted by patients.

5. Reminder System

- A background scheduler runs automatically every 30 minutes.
- The system checks upcoming appointments.
- Reminder emails are sent automatically.
- Duplicate reminders are prevented.

6.1 Conclusion:

The **Dental Clinic Appointment System** successfully provides a digital solution for managing clinic operations efficiently. The system allows patients to book appointments easily and enables the admin to manage appointments, services, and patient records in a structured manner.

The implementation of automated reminder emails (24 hours and 4 hours before appointments) helps reduce missed appointments and improves communication between patients and the clinic. The use of a secure database ensures proper data storage and retrieval.

Overall, the system reduces manual work, minimizes errors, saves time, and improves the productivity of the clinic. This project has demonstrated how modern web technologies can be applied to solve real-world healthcare management problems.

6.1 Further Enhancements:

The system can be improved and expanded in the future with the following enhancements:

1. **Online Payment Integration**
   Allow patients to pay consultation fees online during appointment booking.
2. **SMS Reminder Integration**
   Add SMS notifications along with email reminders.
3. **Patient Login System**
   Provide login access for patients to view appointment history and update details.
4. **Doctor Panel**
   Create a separate dashboard for doctors to manage their schedules.
5. **Reports & Analytics**
   Generate graphical reports for daily, monthly, and yearly appointments.
6. **Multi-Clinic Support**
   Expand the system to manage multiple clinic branches.
7. **Mobile Application**
   Develop an Android/iOS application for easier access.
8. **Rescheduling & Cancellation Feature**
   Allow patients to reschedule or cancel appointments online.

7. Bibliography:

Books:

1. Pressman, R. S. (2014). *Software Engineering: A Practitioner's Approach*. McGraw-Hill Education.
2. Sommerville, I. (2016). *Software Engineering* (10th Edition). Pearson Education.
3. Elmasri, R., & Navathe, S. B. (2016). *Fundamentals of Database Systems* (7th Edition). Pearson.
4. Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7th Edition). McGraw-Hill Education.
5. Duckett, J. (2014). *Web Design with HTML, CSS, JavaScript and jQuery*. Wiley Publications.

Online References:

1. Next.js Official Documentation – https://nextjs.org/docs
2. Node.js Official Documentation – https://nodejs.org
3. PostgreSQL Official Documentation – https://www.postgresql.org/docs
4. Nodemailer Documentation – https://nodemailer.com
5. MDN Web Docs – https://developer.mozilla.org